

쿠버네티스 내부 이해

Understanding Kubernetes Internals

 Kubernetes in Action · Chapter 11

저자: Marko Lukša · 1판 기준 (2판 차이점 포함)

발표자: 박세우

CH 11 11장 미리보기 — 쿠버네티스 안에는 누가 살까?

평소엔 안 보이는 쿠버네티스 내부를 열어, 누가 무슨 일을 하고 서로 어떻게 손발을 맞추는지 하나씩 살펴봐요.

🔗 이 장을 왜 배우나: 쿠버네티스를 "쓰기만" 할 땐 내부를 몰라도 돼요. 하지만 문제가 생겼을 때 "어디가 고장났는지" 짚으려면 내부 부품들의 역할을 알아야 해요. (예: 파드가 계속 Pending이면 → 스케줄러 문제, ImagePull 에러면 → kubelet/런타임 문제)

🍪 쉽게 말하면: 자동차를 "운전"만 하다가 "보닛을 열어 엔진을 들여다보는" 느낌이에요. 엔진·배터리·변속기가 따로 있고 서로 연결돼 있다는 걸 알면, 차가 안 갈 때 어디를 봐야 할지 알게 되죠. 11장이 딱 그 "보닛 열기"예요.

이번 장에서 열어볼 6가지 (카드를 클릭하면 설명이 떠요)

11.1

① 부품 지도 (아키텍처)

쿠버네티스를 이루는 부품들이 뭐가 있고, 두뇌 (컨트롤 플레인)와 일꾼 (워커 노드) 중 어디에 사는지

왜 아키텍처를 이해해야 하는가?

쿠버네티스를 "사용"만 한다면 아키텍처를 몰라도 됩니다. 하지만 운영하거나 트러블슈팅 할 때는 각 컴포넌트의 책임을 명확히 알아야 합니다. 예를 들어 "파드가 Pending 상태에 멈춰있다"면 그건 스케줄러 문제고, "ImagePullBackOff"라면 kubelet/런타임 문제입니다.

핵심 통찰

API 서버가 유일한 통신 허브입니다. 컴포넌트들은 서로 직접 대화하지 않습니다. 모두 API 서버를 통해서만 통신하며, 이게 K8s의 견고함과 확장성의 비밀입니다.

11.2

② 부품들의 팀워크

내가 명령 하나(kubectl apply)를 내리면, 내부에서 부품들이 어떤 순서로 일을 넘겨받는지

이 절에서는 `kubectl apply -f deployment.yaml` 하나가 내부에서 어떻게 여러 컨트롤러를 거쳐 파드 실행까지 이어지는지 추적합니다.

관전 포인트

- Deployment Controller가 Pod를 직접 만들지 않음 → ReplicaSet을 만든다
- ReplicaSet Controller가 Pod를 만들지만 nodeName은 비어있음 → 스케줄러가 채워줌
- 스케줄러도 Pod를 실행하지 않음 → kubelet이 실행
- 모두 Watch를 통한 비동기 협력

11.3

③ 파드의 숨은 비밀

파드 안에 몰래 들어있는 컨테이너(pause)와, 여러 컨테이너가 한 몸처럼 도는 원리

파드는 "함께 실행되는 컨테이너 그룹"이라고 배웠지만, 실제로는 보이지 않는 **pause** 컨테이너가 있어서 네임스페이스를 보유하고 있습니다. 이걸 이해하면 "왜 init container가 끝나도 메인 컨테이너가 같은 네트워크를 쓸 수 있는지", "왜 컨테이너 하나가 죽고 재시작돼도 IP가 안 바뀌는지" 같은 의문이 풀립니다.

11.4

④ 파드끼리 어떻게 대화할까

여기저기 흩어진 파드들이 서로 직접 통신하게 만드는 네트워크 약속과 [CNI](#)

쿠버네티스 네트워크의 황금률: **모든 파드는 NAT 없이 다른 모든 파드와 통신할 수 있어야 한다.** K8s는 이 규칙을 강제하기만 하고, 실제 구현은 **CNI 플러그인**에 맡깁니다. Flannel은 VXLAN 오버레이로, Calico는 BGP 라우팅으로 구현합니다.

11.5

⑤ 안 바뀌는 대표 주소(서비스)

자주 죽고 IP가 바뀌는 파드들을, 안 바뀌는 대표 주소 하나로 묶는 방법

Service의 ClusterIP(예: 10.96.0.10)는 실제 네트워크 인터페이스에 바인딩되어 있지 않습니다. **ip addr**로 검색해도 안 나옵니다. 그런데 어떻게 통신이 되죠?

비밀은 **iptables**입니다. kube-proxy가 노드의 커널에 "이 가상 IP로 가는 트래픽은 실제로는 이 파드들 중 하나로 보내라"는 NAT 규칙을 작성합니다.

11.6

⑥ 하나 죽어도 안 멈추기(HA)

부품이 하나 고장 나도 클러스터 전체가 멈추지 않게 만드는 고가용성 구성

두 종류의 HA

- **애플리케이션 HA**: 우리 앱이 죽지 않도록 → Deployment + 다중 레플리카
- **컨트롤 플레인 HA**: K8s 자체가 죽지 않도록 → etcd 클러스터 + 다중 API 서버

중요한 점: **컨트롤 플레인이 죽어도 실행 중인 워커 노드의 파드는 계속 동작합니다.** kubelet이 캐시된 상태로 자율적으로 운영하기 때문입니다.

📌 **발표 흐름**: 먼저 **부품을 하나씩(11.1)** 소개하고 → 그 부품들이 **어떻게 협력하는지(11.2)** 본 다음 → 파드·네트워크·서비스·고가용성(11.3~11.6) 순으로 깊이 들어가요. 길을 잃으면 왼쪽 목차를 보세요!

이 슬라이드의 용어 풀이

네임스페이스

여기서는 리눅스 커널의 격리 기능이에요. 프로세스마다 네트워크·PID 등을 따로 보이게 해서 컨테이너를 서로 격리해요. (K8s의 namespace 리소스와는 다른 개념!)

스케줄러

"이 파드를 어느 노드에서 실행할까?"만 결정하는 컴포넌트예요. 파드를 직접 만들거나 실행하지는 않아요. 조건에 맞는 노드를 고르는 게 전부예요.

워커 노드

실제로 앱 컨테이너가 돌아가는 일꾼 서버예요. 컨트롤 플레인인 "이 파드를 띄워"라고 시키면 여기서 실행돼요.

컨트롤 플레인

클러스터의 "두뇌" 역할을 하는 영역이에요. 무엇을 어디서 실행할지 결정하는 관리 컴포넌트(API 서버·etcd·스케줄러·컨트롤러 매니저)가 모여 있어요. 예전엔 "마스터 노드"라고 불렀어요.

쿼럼

"과반수"라는 뜻이에요. 예: 3대 중 2대가 동의해야 결정이 확정돼요. 그래서 etcd는 홀수 개(3,5,7)로 두어 과반을 명확히 만 들어요.

API 서버

모든 컴포넌트가 거쳐가는 중앙 창구예요. kubectl·스케줄러·kubelet 등은 서로 직접 대화하지 않고 전부 API 서버를 통해서만 통신해요. (영문: API Server)

BGP

인터넷 라우터들이 "이 IP 대역은 내 쪽으로 와"라고 경로를 알리는 프로토콜이에요. Calico는 이걸 써서 캡슐화 없이 노드 간 파드 통신을 라우팅해요.

ClusterIP

Service의 기본 유형. 클러스터 "내부에서만" 접근 가능한 가상 IP를 줘요. 실제 서버가 아니라 iptables 규칙으로 만들어진 가짜 IP예요.

CNI

Container Network Interface. kubelet이 파드를 만들 때 "IP 할당, 네트워크 연결"을 위임하는 플러그인 표준이에요. Calico·Cilium·Flannel 등이 구현체예요.

etcd

클러스터의 모든 정보(어떤 파드가 있고 상태가 어떤지 등)를 저장하는 데이터베이스예요. 클러스터의 "공식 장부"라고 보면 돼요. 오직 API 서버만 직접 접근할 수 있어요.

iptables

리눅스 커널의 패킷 필터링/변환 규칙 시스템이에요. kube-proxy가 여기에 규칙을 넣어 Service IP로 온 트래픽을 실제 파드로 보내요.

kube-proxy

각 노드에서 Service(가상 IP)로 온 트래픽을 실제 파드로 전달되도록 네트워크 규칙(iptables 등)을 관리하는 컴포넌트예요.

pause 컨테이너

파드마다 몰래 하나씩 뜨는 거의 아무것도 안 하는 컨테이너예요. 파드의 네트워크/IPC 공간을 처음 만들어 "붙잡고" 있어서, 앱 컨테이너가 재시작돼도 파드 IP가 유지돼요.

VXLAN

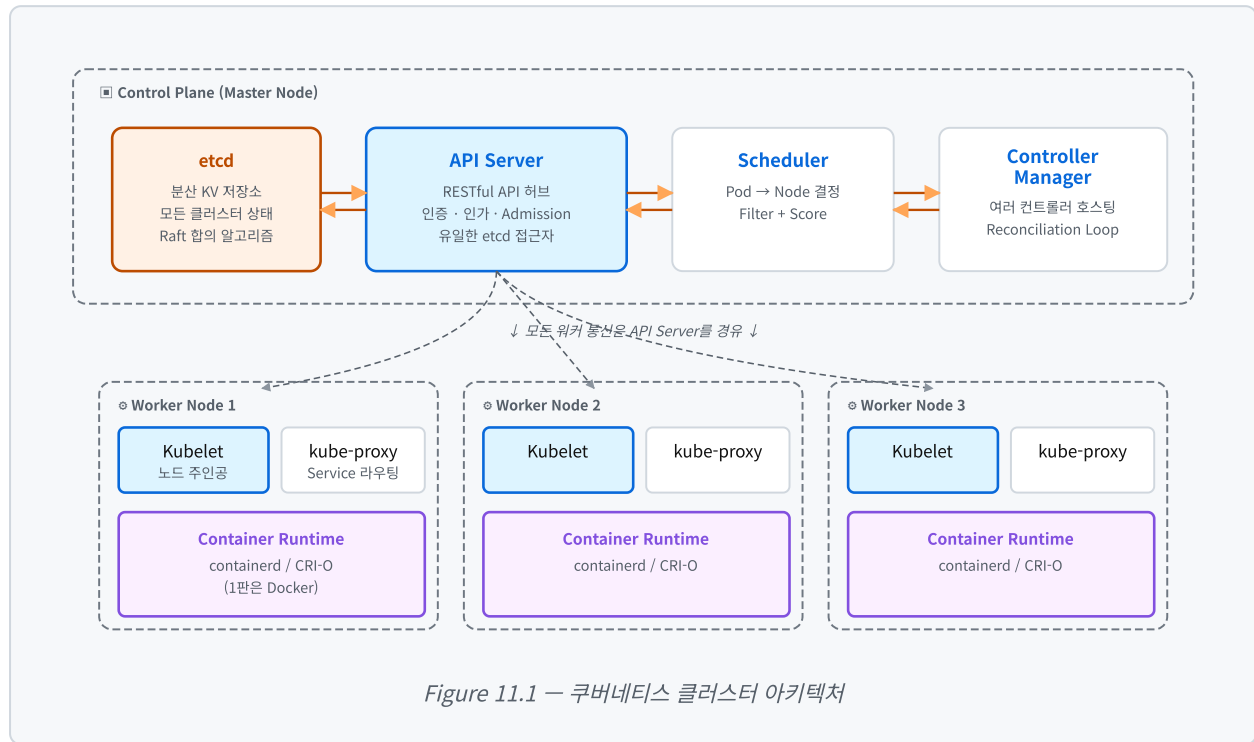
파드 패킷을 UDP로 한 번 감싸서 노드 사이를 건너게 하는 오버레이 기술이에요. Flannel의 기본 방식이에요.

Watch

API 서버가 "리소스가 바뀌면 알려줄게"라고 변경 사항을 실시간으로 밀어주는(push) 기능이에요. 계속 물어보는 폴링보다 효율적이에요.

11.1 아키텍처 이해

쿠버네티스 클러스터는 크게 컨트롤 플레인 (마스터)과 워커 노드로 나뉩니다.



마스터 노드 컴포넌트

[etcd](#) · [API Server](#) · [Scheduler](#) · [Controller Manager](#)

① etcd

분산 KV 저장소. 모든 클러스터 상태가 여기 저장됩니다.

② API Server (kube-apiserver)

RESTful API를 제공하는 게이트웨이. 모든 컴포넌트가 여기를 통해 통신합니다.

③ Scheduler (kube-scheduler)

새 파드를 적절한 노드에 할당합니다. Bind만 하고 실행은 kubelet이 합니다.

④ Controller Manager (kube-controller-manager)

여러 컨트롤러를 호스팅하는 단일 프로세스. 각 컨트롤러는 reconciliation loop를 돌립니다.

💡 이 4가지가 다 죽어도 워커 노드의 파드는 계속 실행됩니다. 단지 "변화"가 일어나지 않을 뿐입니다.

워커 노드 컴포넌트

[Kubelet](#) · [kube-proxy](#) · [Container Runtime](#)

① Kubelet

노드의 "주인공". 자신에게 할당된 파드를 실행/감시/보고합니다.

② kube-proxy

서비스 트래픽을 실제 파드로 전달하는 네트워크 프록시. iptables/IPVS 규칙을 관리합니다.

③ Container Runtime

실제 컨테이너를 실행하는 엔진. 1판은 Docker, 최신은 containerd/CRI-O가 표준입니다.

💡 Kubelet은 systemd로 실행되는 "진짜 프로세스"인 반면, 다른 컴포넌트들(API 서버, 컨트롤러 매니저 등)은 보통 컨트롤 플레인에서 **static pod**로 실행됩니다.

11.1.1

분산 시스템의 본질

각 컴포넌트는 독립 프로세스, 실패 가능성 항상 고려

분산 시스템이 의미하는 것

- 각 컴포넌트는 **독립 프로세스**로 실행됩니다 (보통 다른 노드에서)
- 네트워크는 언제든지 끊길 수 있고, 컴포넌트는 언제든지 죽을 수 있다고 **가정**합니다
- 그래서 모든 컴포넌트는 **재시작 가능**해야 하고, 상태는 **etcd**에만 저장됩니다

왜 이런 설계가 좋은가?

컴포넌트 하나가 죽어도 다른 컴포넌트는 영향받지 않고, 죽었던 컴포넌트가 재시작되면 etcd에서 상태를 다시 읽어 이어서 일합니다. 이게 K8s의 **견고함 (resilience)**의 비밀입니다.

2판 변화

컨트롤러 매니저 분리

1판에서는 통합된 `kube-controller-manager` 로 설명하지만, 최신 버전에서는 클라우드 관련 컨트롤러가 `cloud-controller-manager` 로 분리되었습니다. 이를 통해 코어 K8s와 클라우드 제공자 코드가 독립적으로 발전할 수 있습니다.

이 슬라이드의 용어 풀이

워커 노드

실제로 앱 컨테이너가 돌아가는 일꾼 서버예요. 컨트롤 플레인인 "이 파드를 띄워"라고 시키면 여기서 실행돼요.

인가

"그걸 할 권한 있어?"를 확인하는 단계예요. (영문: Authorization) 신원이 확인된 뒤, 그 사람이 해당 작업을 해도 되는지 RBAC 등으로 검사해요.

인증

"너 누구야?"를 확인하는 단계예요. (영문: Authentication) 인증서·토큰·ServiceAccount 등으로 요청자의 신원을 확인해요.

컨트롤 플레인

클러스터의 "두뇌" 역할을 하는 영역이에요. 무엇을 어디서 실행할지 결정하는 관리 컴포넌트(API 서버·etcd·스케줄러·컨트롤러 매니저)가 모여 있어요. 예전엔 "마스터 노드"라고 불렀어요.

컨트롤러 매니저

여러 "컨트롤러"를 한데 모아 돌리는 프로세스예요. 각 컨트롤러는 "원하는 상태"와 "현재 상태"를 비교해 계속 맞춰주는 일을 해요.

API 서버

모든 컴포넌트가 거쳐가는 중앙 창구예요. kubectl·스케줄러·kubelet 등은 서로 직접 대화하지 않고 전부 API 서버를 통해서만 통신해요. (영문: API Server)

API Server

모든 컴포넌트가 거쳐가는 중앙 창구예요. 서로 직접 대화하지 않고 전부 여기를 통해서만 통신해요. etcd에 접근하는 유일한 통로이기도 해요.

Container Runtime

컨테이너를 실제로 실행하는 엔진이에요. kubelet의 지시를 받아 진짜로 컨테이너를 띄워요. 요즘은 containerd나 CRI-O를 써요.

containerd

가장 널리 쓰이는 컨테이너 런타임이에요. 원래 Docker 내부에서 쓰던 엔진이 독립한 것. Docker로 만든 이미지도 그대로 실행돼요.

Controller Manager

여러 컨트롤러를 한데 모아 돌리는 프로세스예요. 각 컨트롤러는 "원하는 상태"와 "현재 상태"를 끊임없이 비교해 맞춰줘요.

CRI

Container Runtime Interface. kubelet과 컨테이너 런타임 사이의 "표준 약속"이에요. 이 규격만 맞추면 어떤 런타임이든 K8s에 끼울 수 있어요.

CRI-O

쿠버네티스 전용으로 가볍게 설계된 컨테이너 런타임이에요. OpenShift의 기본 런타임이에요.

etcd

클러스터의 모든 정보(어떤 파드가 있고 상태가 어떤지 등)를 저장하는 데이터베이스예요. 클러스터의 "공식 장부"라고 보면 돼요. 오직 API 서버만 직접 접근할 수 있어요.

iptables

리눅스 커널의 패킷 필터링/변환 규칙 시스템이에요. kube-proxy가 여기에 규칙을 넣어 Service IP로 온 트래픽을 실제 파드로 보내요.

IPVS

리눅스 커널의 로드밸런싱 기능이에요. iptables보다 규칙이 많아도 빨라서(O(1)) 서비스가 수천 개인 대규모 환경에 권장돼요.

kube-proxy

각 노드에서 Service(가상 IP)로 온 트래픽을 실제 파드로 전달되도록 네트워크 규칙(iptables 등)을 관리하는 컴포넌트예요.

Kubelet

각 워커 노드의 주인공 에이전트예요. API 서버에서 "이 노드에 배정된 파드"를 받아, 컨테이너 런타임에게 시켜 실제로 컨테이너를 띄우고 상태를 보고해요.

Raft

etcd 같은 분산 시스템에서 여러 서버가 "같은 데이터"에 합의하도록 만드는 알고리즘이에요. 다수결로 리더를 뽑고, 그 리더가 기록을 결정해요.

Reconciliation Loop

"조정 루프"예요. 컨트롤러가 "내가 원하는 상태(예: 파드 3개)"와 "지금 실제 상태"를 비교해서, 다를 때마다 같아지도록 계속 고치는 무한 반복 동작이에요.

RESTful

HTTP의 GET/POST/PUT/DELETE로 리소스를 다루는 표준 API 설계 방식이에요. K8s API 서버도 이 방식이라 kubectl이 결국 HTTP 요청을 보내는 거예요.

Scheduler

"이 파드를 어느 노드에서 실행할까?"만 결정해요. 파드를 직접 만들거나 실행하지는 않고, 조건에 맞는 노드를 골라주는 역할이에요.

11.1.2 etcd: 클러스터 상태 저장소

쿠버네티스의 모든 상태가 저장되는 분산 키-값 저장소

🎯 **한 줄 결론:** etcd는 클러스터의 "진짜 데이터"가 사는 **유일한 장소**예요. 어떤 파드가 있고 상태가 어떤지 등 모든 정보가 여기에만 있고, 나머지 컴포넌트는 전부 여기 적힌 걸 보고 행동해요.

🍔 **쉽게 말하면:** 회사의 공식 장부(금고)예요. 모두 이 장부를 기준으로 일하죠. 장부가 손상되면 회사가 마비 되니까, 똑같은 사본을 여러 권(홀수 개) 두고 과반이 일치할 때만 기록을 인정해요. 그래야 한 권이 고장 나도 안전하니까요.

핵심 특징

- ▶ **분산 KV 스토어** · Raft 합의 알고리즘 사용
- ▶ **강한 일관성(strong consistency)** 보장
- ▶ **홀수 개** 인스턴스 권장 (3, 5, 7 - 쿼럼 확보)
- ▶ **API 서버만** etcd와 직접 통신
- ▶ **낙관적 동시성 제어** (resourceVersion 활용)

저장되는 데이터

- ▶ Pod, Service, Deployment 등 **모든 리소스** 매니페스트
- ▶ 리소스의 **현재 상태**와 **이벤트**
- ▶ 인증/인가 관련 **ConfigMap, Secret**
- ▶ JSON 또는 protobuf 형식으로 직렬화

알고리즘

Raft 합의 알고리즘

왜 홀수 개? 왜 쿼럼이 필요한가? Split-brain 방지 원리

쿼럼(Quorum)이란?

쿼럼은 "다수결을 위한 최소 인원"입니다. N개 노드에서 쿼럼은 $(N/2) + 1$ 입니다.

- 3개 노드 → 쿼럼 2 → 1개 죽어도 OK
- 4개 노드 → 쿼럼 3 → 1개 죽어도 OK (4개나 5개나 똑같이 1개만 견딤!)
- 5개 노드 → 쿼럼 3 → 2개 죽어도 OK

왜 짝수가 나쁜가?

짝수 개는 **장애 내성은 같은데 비용만 더 든다**는 게 핵심입니다. 게다가 네트워크 분할 시 정확히 절반씩 나뉘면 **split-brain** 위험이 있습니다.

Raft가 보장하는 것

리더 1명이 모든 쓰기를 처리하고, 쓰기는 **쿼럼이 동의해야** 커밋됩니다. 이게 강한 일관성(strong consistency)의 근거입니다.

동시성

낙관적 동시성 제어

resourceVersion이 변경 충돌을 어떻게 막는가

문제 상황

컨트롤러 A와 B가 동시에 같은 Pod를 수정하려고 한다면? 한쪽 변경이 사라질 수 있습니다.

해결책: resourceVersion

모든 리소스에는

`metadata.resourceVersion`이 있고, 이게 변경될 때마다 증가합니다.

1. A가 Pod를 읽음 (resourceVersion=100)
 2. A가 수정해서 PUT 요청 (resourceVersion=100 포함)
 3. B가 먼저 수정해서 resourceVersion이 101이 됨
 4. A의 PUT은 **409 Conflict** 반환 → A는 다시 읽고 재시도
- 이게 "**잠금 없이도**" 동시성 문제를 해결하는 방식입니다.

2판

etcd v2 → v3 전환

1판은 v2/v3 혼재. 최신은 v3 전용 ([gRPC](#), 더 효율적)

etcd v2 (1판 시절)

- HTTP+JSON 기반 REST API
- 디렉토리 구조처럼 키 저장
- 변경 이력(history)이 limited

etcd v3 (현재 표준)

- gRPC + protobuf 기반 (훨씬 빠름)
- 플랫 키 공간 (디렉토리 X)
- MVCC 기반 (모든 변경 이력 보존)
- Lease 기반 TTL (이벤트, 리스 관리에 좋음)

K8s에서의 변화

K8s 1.13에서 etcd v3가 기본, 1.22에서 v2 지원 완전 제거. 마이그레이션이 필요했지만 지금은 모든 클러스터가 v3입니다.

etcd v3 전용 및 보안 강화

K8s 1.22+ 부터 etcd v2가 완전히 제거되었습니다. v3는 gRPC 기반으로 더 빠르며 lease 기반 TTL을 제공합니다. 또한 `EncryptionConfiguration` 으로 etcd 내 Secret 암호화가 표준이 되었습니다.

이 슬라이드의 용어 풀이

강한 일관성

어느 서버에 물어봐도 항상 같은 최신 값을 돌려준다는 보장이예요. (영문: strong consistency) 클러스터 상태가 서버마다 달라지면 안 되니까 etcd는 이걸 보장해요.

낙관적 동시성 제어

"일단 충돌이 잘 안 날 거다"라고 가정하고, 저장할 때 버전 번호를 확인해 그 사이 누가 바꿨으면 거부하는 방식이에요. 잠금(lock) 없이 충돌만 막아요.

인가

"그걸 할 권한 있어?"를 확인하는 단계예요. (영문: Authorization) 신원이 확인된 뒤, 그 사람이 해당 작업을 해도 되는지 RBAC 등으로 검사해요.

인증

"너 누구야?"를 확인하는 단계예요. (영문: Authentication) 인증서·토큰·ServiceAccount 등으로 요청자의 신원을 확인해요.

쿼럼

"과반수"라는 뜻이에요. 예: 3대 중 2대가 동의해야 결정이 확정돼요. 그래서 etcd는 홀수 개(3,5,7)로 두어 과반을 명확히 만 들어요.

API 서버

모든 컴포넌트가 거쳐가는 중앙 창구예요. kubectl·스케줄러·kubelet 등은 서로 직접 대화하지 않고 전부 API 서버를 통해서만 통신해요. (영문: API Server)

ConfigMap

앱 설정값(환경변수, 설정파일 등)을 코드와 분리해 담아두는 리소스예요. 비밀이 아닌 일반 설정에 써요.

etcd

클러스터의 모든 정보(어떤 파드가 있고 상태가 어떤지 등)를 저장하는 데이터베이스예요. 클러스터의 "공식 장부"라고 보면 돼요. 오직 API 서버만 직접 접근할 수 있어요.

gRPC

구글이 만든 빠른 통신 방식이에요. 일반 REST(JSON)보다 가볍고 효율적이라 etcd v3가 이걸 써요.

Lease

"내가 지금 리더야"를 잠깐 동안 빌려 표시하는 리소스예요. 주기적으로 갱신하고, 끊기면 다른 인스턴스가 가져가요. Leader Election에 써요.

protobuf

protocol buffers의 줄임말. 데이터를 사람이 읽는 JSON 대신 작고 빠른 바이너리로 압축해 주고받는 형식이에요.

Raft

etcd 같은 분산 시스템에서 여러 서버가 "같은 데이터"에 합의하도록 만드는 알고리즘이에요. 다수결로 리더를 뽑고, 그 리더가 기록을 결정해요.

resourceVersion

모든 리소스가 가진 "버전 번호"예요. 수정할 때 이 번호가 그대로면 OK, 그 사이 누가 바뀌서 번호가 달라졌으면 충돌로 거부해요. 낙관적 동시성 제어의 핵심이에요.

Secret

비밀번호·토큰·인증서 같은 민감한 값을 담는 리소스예요. ConfigMap과 비슷하지만 민감 정보용이에요.

11.1.3 API 서버의 역할

모든 컴포넌트가 통신하는 중앙 허브. RESTful API를 제공합니다.

🎯 **한 줄 결론:** 클러스터의 **모든 요청은 반드시 API 서버를 거쳐요**. etcd에 접근할 수 있는 유일한 문지기죠. 들어온 요청을 4단계로 검문한 뒤에야 저장해요.

🍔 **쉽게 말하면:** 회사 정문의 **안내데스크**예요. ① 너 누구야?(인증) → ② 들어갈 권한 있어?(인가) → ③ 반입 금지품 없어?(정책 검사/Admission) → ④ 서류 양식 맞아?(검증) 이걸 다 통과해야 안으로(etcd) 들여보내 줘요.



Figure 11.2 — API Server 요청 처리 파이프라인

단계 1

인증 (Authentication)

클라이언트 인증서, Bearer 토큰, [ServiceAccount](#) 등

지원되는 인증 방식

- **클라이언트 인증서**: kubeconfig에 들어있는 cert/key 페어 (kubectl 기본)
- **Bearer 토큰**: Authorization 헤더에 토큰
- **ServiceAccount 토큰**: 파드 안에서 사용하는 JWT
- **OIDC**: 외부 IDP 연동 (구글, AAD 등)
- **Webhook**: 외부 서비스에 인증 위임

결과

인증이 성공하면 **사용자명(username)**과 **그룹(groups)**이 결정됩니다. 이게 다음 단계인 인가에 사용됩니다.

단계 2

인가 (Authorization)

[RBAC](#), ABAC, Webhook 등으로 권한 확인

RBAC (Role-Based Access Control)

가장 널리 쓰이는 인가 방식입니다.

- **Role / ClusterRole**: 권한 묶음 정의 (예: "pods에 대한 get/list")
- **RoleBinding / ClusterRoleBinding**: 사용자/SA에 Role 부여

인가 흐름

요청 정보(user, verb, resource, namespace)와 등록된 RBAC 규칙을 매칭해서 허용/거부를 결정합니다.

💡 `kubectl auth can-i create pods`로 권한을 미리 확인할 수 있습니다.

단계 3

[Admission Control](#)

리소스 검증/변경 플러그인 (ResourceQuota, LimitRanger 등)

두 종류의 Admission

- **Mutating Admission**: 리소스를 수정합니다 (예: 기본값 주입, 사이드카 자동 추가)
- **Validating Admission**: 리소스를 검증만 합니다 (통과 또는 거부)

내장 컨트롤러 예시

- **ResourceQuota**: 네임스페이스 쿼터 확인
- **LimitRanger**: 기본 리소스 limit 주입
- **ServiceAccount**: 파드에 SA 토큰 마운트
- **PodSecurity**: PSA 적용 (K8s 1.25+)

커스텀: Admission Webhook

외부 서비스를 Admission으로 등록할 수 있습니다. OPA Gatekeeper, Kyverno 등이 이걸 활용합니다.

2판 변화

PodSecurityPolicy 제거 → Pod Security Admission

1판에 나오는 **PodSecurityPolicy(PSP)** 는 K8s 1.21에서 deprecated, 1.25에서 완전 제거되었습니다. 대신 **Pod Security Admission(PSA)** 이 내장 어드미션 컨트롤러로 들어왔으며, **Privileged / Baseline / Restricted** 3단계 표준을 namespace 레이블로 적용합니다.

이 슬라이드의 용어 풀이

네임스페이스

여기서는 리눅스 커널의 격리 기능이에요. 프로세스마다 네트워크·PID 등을 따로 보이게 해서 컨테이너를 서로 격리해요. (K8s의 namespace 리소스와는 다른 개념!)

스케줄러

"이 파드를 어느 노드에서 실행할까?"만 결정하는 컴포넌트예요. 파드를 직접 만들거나 실행하지는 않아요. 조건에 맞는 노드를 고르는 게 전부예요.

인가

"그걸 할 권한 있어?"를 확인하는 단계예요. (영문: Authorization) 신원이 확인된 뒤, 그 사람이 해당 작업을 해도 되는지 RBAC 등으로 검사해요.

인증

"너 누구야?"를 확인하는 단계예요. (영문: Authentication) 인증서·토큰·ServiceAccount 등으로 요청자의 신원을 확인해요.

Admission Control

인증·인가를 통과한 요청을 etcd에 저장하기 직전에 "정책 검사/자동 수정"하는 단계예요. 예: 리소스 한도 초과 막기, 기본값 채워주기 등.

API 서버

모든 컴포넌트가 거쳐가는 중앙 창구예요. kubectl·스케줄러·kubelet 등은 서로 직접 대화하지 않고 전부 API 서버를 통해서만 통신해요. (영문: API Server)

API Server

모든 컴포넌트가 거쳐가는 중앙 창구예요. 서로 직접 대화하지 않고 전부 여기를 통해서만 통신해요. etcd에 접근하는 유일한 통로이기도 해요.

etcd

클러스터의 모든 정보(어떤 파드가 있고 상태가 어떤지 등)를 저장하는 데이터베이스예요. 클러스터의 "공식 장부"라고 보면 돼요. 오직 API 서버만 직접 접근할 수 있어요.

OIDC

OpenID Connect. 구글 로그인 같은 외부 신원 제공자로 K8s에 로그인하게 해주는 인증 표준이에요.

RBAC

Role-Based Access Control. "역할(Role)"에 권한을 묶고, 사용자/서비스에 역할을 부여해 권한을 관리하는 방식이에요. K8s의 표준 인가 방법이에요.

RESTful

HTTP의 GET/POST/PUT/DELETE로 리소스를 다루는 표준 API 설계 방식이에요. K8s API 서버도 이 방식이라 kubectl이 결국 HTTP 요청을 보내는 거예요.

ServiceAccount

사람이 아니라 "파드(앱)"가 API 서버에 접근할 때 쓰는 계정이에요. 파드 안에 토큰이 자동으로 들어가요.

Watch

API 서버가 "리소스가 바뀌면 알려줄게"라고 변경 사항을 실시간으로 밀어주는(push) 기능이에요. 계속 물어보는 폴링보다 효율적이에요.

11.1.4 Watch 메커니즘

API 서버는 리소스 변경을 클라이언트에게 푸시(push) 합니다. 폴링이 아닙니다.

🎯 **한 줄 결론:** 컴포넌트들이 "뭐 바뀐 거 있어?"를 계속 캐묻지 않아요. **변화가 생기면 API 서버가 먼저 알려 줘요(push)**. 이 "알림 받기"가 쿠버네티스 자동화 전체를 굴리는 엔진이에요.

🍔 **쉽게 말하면:** 유튜브 구독 알림이랑 같아요. 매번 채널에 들어가 새 영상 났나 확인(폴링)하면 비효율적이죠. 구독(Watch)해두면 새 영상이 올라오는 순간 알림이 오잖아요. 스케줄러·컨트롤러·kubelet이 다 이 알림을 받고 움직여요.

왜 Watch가 중요한가?

- ▶ 스케줄러, 컨트롤러, kubelet 등은 모두 **변경을 감지**해서 동작합니다.
- ▶ 주기적 폴링(polling)은 비효율적 → HTTP long-polling 또는 streaming 방식 사용
- ▶ 변경 이벤트는 **ADDED, MODIFIED, DELETED** 타입으로 전달

```
# Watch API 사용 예시
GET /api/v1/pods?watch=true&resourceVersion=12345

# 응답 스트림
{"type":"ADDED", "object":{...pod manifest...}}
{"type":"MODIFIED","object":{...updated pod...}}
{"type":"DELETED", "object":{...deleted pod...}}
```


메커니즘

Long-polling vs Streaming

HTTP 연결 유지하며 변경을 푸시하는 원리

Watch가 폴링보다 좋은 이유

- 변경이 없으면 트래픽이 없음 (효율적)
- 변경 직후 즉시 알림 (지연 최소화)
- resourceVersion 기반이라 **이벤트 누락이 없음**

재연결 시 처리

Watch 연결이 끊기면 클라이언트는 마지막으로 받은 resourceVersion을 가지고 재연결합니다. API 서버는 그 이후 변경들을 다시 스트리밍합니다. 단, 너무 오래된 resourceVersion은 **"410 Gone"**으로 거부될 수 있고, 이때는 **List + Watch**를 다시 해야 합니다.

패턴

Informer 패턴

client-go의 Informer가 Watch를 어떻게 추상화하는가

Informer가 하는 일

1. 처음에 **List**로 전체 리소스 가져오기
2. 이후 **Watch**로 변경 사항 받기
3. 로컬 메모리에 **캐시(인덱스)** 유지
4. 변경 이벤트 핸들러 호출 (Add/Update/Delete)

왜 이게 중요한가?

모든 컨트롤러가 직접 API 서버에 매번 물어보면 API 서버가 폭발합니다. Informer가 캐시 + Watch로 **최소한의 API 호출**로 최신 상태를 유지합니다.

💡 컨트롤러를 직접 만들 때 (Operator 개발 등) controller-runtime 또는 client-go의 Informer를 사용합니다.

💡 **발표 포인트:** 쿠버네티스의 모든 자동화는 "선언된 상태"와 "현재 상태"의 차이를 Watch로 감지해서 reconciliation 하는 것입니다. 이 원리를 이해하면 K8s가 왜 그렇게 견고한지 알 수 있습니다.

이 슬라이드의 용어 풀이

스케줄러

"이 파드를 어느 노드에서 실행할까?"만 결정하는 컴포넌트예요. 파드를 직접 만들거나 실행하지는 않아요. 조건에 맞는 노드를 고르는 게 전부예요.

API 서버

모든 컴포넌트가 거쳐가는 중앙 창구예요. kubectl·스케줄러·kubelet 등은 서로 직접 대화하지 않고 전부 API 서버를 통해서만 통신해요. (영문: API Server)

Informer

K8s 클라이언트 라이브러리의 핵심 도구예요. Watch로 받은 데이터를 로컬에 캐시해두고 변경 이벤트를 처리해줘서, 컨트롤러를 쉽게 만들 수 있게 해줘요.

Long-polling

클라이언트가 요청을 보내고 응답을 바로 끊지 않은 채 "변화가 생길 때까지" 연결을 유지하는 방식이에요. 변화가 생기면 그때 응답이 와요.

Operator

CRD + 커스텀 컨트롤러로, 사람이 하던 운영 작업(DB 백업·복구 등)을 자동화하는 패턴이에요. K8s를 무한 확장하는 방법이에요.

resourceVersion

모든 리소스가 가진 "버전 번호"예요. 수정할 때 이 번호가 그대로면 OK, 그 사이 누가 바뀌서 번호가 달라졌으면 충돌로 거부해요. 낙관적 동시성 제어의 핵심이에요.

streaming

한 번 연결을 열어두면, 그 연결로 변경 사항을 **여러 번 계속** 흘려보내는 방식이에요. long-polling은 "한 번 응답 주고 다시 연결"을 반복하지만, streaming은 연결을 끊지 않고 ADDED/MODIFIED/DELETED 이벤트를 계속 밀어줘요. (예: 라디오 방송처럼 채널만 맞춰두면 계속 들리는 느낌) 쿠버네티스 Watch가 이 방식이에요.

Watch

API 서버가 "리소스가 바뀌면 알려줄게"라고 변경 사항을 실시간으로 밀어주는(push) 기능이에요. 계속 물어보는 폴링보다 효율적이에요.

11.1.5 스케줄러

새로 생성된 파드를 어느 노드에 배치할지 결정합니다.

🎯 **한 줄 결론:** 스케줄러는 "**이 파드를 어느 노드에서 돌릴까?**"만 정해요. 파드를 직접 만들거나 실행하지 않아요. ① 조건 안 맞는 노드를 거르고(Filter) → ② 남은 노드에 점수를 매겨(Score) → 1등 노드를 골라줄 뿐이에요.

🍔 **쉽게 말하면:** 식당의 **자리 안내 호스트**예요. 손님(파드)을 빈 테이블(노드)로 안내만 하죠. 만석인 자리는 빼고(Filter), 남은 자리 중 가장 좋은 곳으로(Score) 안내해요. **요리(컨테이너 실행)는 주방(kubelet)이** 하고요.

스케줄러의 동작 방식



단계 1

Filtering (Predicates)

리소스 요구사항, nodeSelector, taint 등으로 노드 후보 압축

주요 Filter (옛 Predicate) 예시

- **NodeResourcesFit**: requests를 감당할 CPU/메모리가 있는가
- **NodeSelector**: pod의 nodeSelector와 node label이 일치하는가
- **TaintToleration**: 노드의 taint를 파드가 tolerate 하는가
- **VolumeBinding**: PVC가 바인딩 가능한 영역의 노드인가
- **NodeAffinity**: nodeAffinity 규칙 만족하는가
- **PodTopologySpread**: 토폴로지 분산 제약 만족하는가

모든 Filter를 통과한 노드만 다음 Scoring 단계로 넘어갑니다. 통과한 노드가 0개면 Pod는 **Pending** 상태로 남습니다.

단계 2

Scoring (Priorities)

남은 노드에 점수를 매기고 최고점 노드 선택

주요 Score 플러그인

- **NodeResourcesBalancedAllocation**: CPU/메모리가 균형 잡힌 노드 선호
- **ImageLocality**: 이미지가 이미 캐시된 노드 선호 (pull 시간 절약)
- **InterPodAffinity**: pod affinity/anti-affinity 점수
- **TaintToleration**: PreferNoSchedule taint는 점수 감점
- **NodeAffinity**: preferred(soft) nodeAffinity 점수

각 플러그인이 0~100점을 매기고, 가중치 합산 후 최고점 노드가 선택됩니다. 동점이면 랜덤.

고급

Affinity / Anti-Affinity

파드를 특정 노드/파드 근처(혹은 멀리) 배치

NodeAffinity

특정 노드에 배치하려는 경우. nodeSelector의 표현력이 풍부한 버전.

- `requiredDuringSchedulingIgnoredDuringExecution`: 필수 (hard)
- `preferredDuringSchedulingIgnoredDuringExecution`: 선호 (soft)

PodAffinity / PodAntiAffinity

- **PodAffinity**: 다른 파드 근처에 배치 (예: 캐시 파드 옆에)
- **PodAntiAffinity**: 다른 파드와 멀리 (예: 같은 앱 레플리카 분산)

실전 패턴

같은 Deployment의 레플리카가 한 노드에 몰리지 않도록 **PodAntiAffinity** + **topologyKey=hostname**을 자주 씁니다. HA의 기본기입니다.

2판 변화

스케줄링 프레임워크 (Scheduling Framework)

K8s 1.15+ 부터 **플러그인 기반 스케줄링 프레임워크**가 도입되었습니다. `PreFilter`, `Filter`, `PostFilter`, `Score`, `Reserve`, `Permit`, `PreBind`, `Bind` 등의 확장 포인트가 있어, 커스텀 스케줄링 로직을 쉽게 끼워 넣을 수 있습니다. 1판의 "Predicate/Priority"는 옛 명칭이며, 지금은 모두 플러그인입니다.

이 슬라이드의 용어 풀이

스케줄러

"이 파드를 어느 노드에서 실행할까?"만 결정하는 컴포넌트예요. 파드를 직접 만들거나 실행하지는 않아요. 조건에 맞는 노드를 고르는 게 전부예요.

인가

"그걸 할 권한 있어?"를 확인하는 단계예요. (영문: Authorization) 신원이 확인된 뒤, 그 사람이 해당 작업을 해도 되는지 RBAC 등으로 검사해요.

API Server

모든 컴포넌트가 거쳐가는 중앙 창구예요. 서로 직접 대화하지 않고 전부 여기를 통해서만 통신해요. etcd에 접근하는 유일한 통로이기도 해요.

11.1.6 컨트롤러 매니저

여러 컨트롤러를 호스팅하는 단일 프로세스. 각 컨트롤러는 "원하는 상태"를 유지합니다.

🎯 **한 줄 결론:** 컨트롤러는 "내가 원하는 상태(예: 파드 3개)"와 "지금 실제 상태"를 끊임없이 비교해서, 다른 것 같아지도록 계속 고쳐요(조정 루프). 컨트롤러 하나는 딱 한 가지 일만 맡아요.

🍔 **쉽게 말하면:** 에어컨 온도조절기랑 똑같아요. 설정 26도(원하는 상태)와 현재 온도를 계속 비교해서, 더우면 켜고 도달하면 멈추죠. "파드 3개 유지"도 마찬가지로요. 2개로 줄면 1개 더 만들고, 4개가 되면 1개 지워요.

리컨실리에이션 루프 (Reconciliation Loop)

```
for {  
    // 1. 현재 상태 관찰 (Watch)  
    desired := getDesiredState() // e.g. replicas: 3  
    actual   := getActualState()  // e.g. running pods: 2  
  
    // 2. 차이 계산  
    diff := desired - actual  
  
    // 3. 조정 액션  
    if diff > 0 { createPod() }  
    if diff < 0 { deletePod() }  
}
```

주요 컨트롤러들

워크로드

Replication Manager

RC/RS가 정의한 레플리카 수 유지

"replicas: 3"이라고 선언된 RC를 보고, 실제 동작 중인 파드 수를 세서 차이를 메꾸는 단순한 컨트롤러입니다.

동작

- 현재 매치되는 파드 수 < replicas → 부족한 만큼 Pod 생성
- 현재 매치되는 파드 수 > replicas → 초과한 만큼 Pod 삭제
- Pod 삭제 우선순위: 준비 안된 파드 > 짧은 시간 실행된 파드 > ...

실무에서는 RC 대신 **ReplicaSet**(label selector가 더 강력)이나 Deployment를 씁니다.

워크로드

Deployment 컨트롤러

ReplicaSet을 만들고 롤링 업데이트 수행

책임

- Deployment 스펙 변경 시 → 새 ReplicaSet 생성
- 롤링 업데이트 진행 (구 RS의 replicas--, 신 RS의 replicas++)
- 롤백 시 → 이전 ReplicaSet의 replicas를 다시 늘림
- maxSurge/maxUnavailable 제어

중요한 사실

Deployment 컨트롤러는 **Pod를 직접 만들지 않습니다**. ReplicaSet을 만들고, 그 ReplicaSet의 컨트롤러가 Pod를 만듭니다. 책임 분리의 좋은 예시입니다.

노드

Node 컨트롤러

노드 상태 모니터링, 응답 없는 노드의 파드 제거

역할

- kubelet의 하트비트(NodeStatus 업데이트) 모니터링
- 응답 없는 노드는 **NotReady** 표시
- 일정 시간(기본 5분) 후에도 응답 없으면 해당 노드의 파드 evict
- NoExecute taint 자동 추가

주의

노드가 진짜 죽은 건지, 네트워크만 끊긴 건지 구별이 어렵습니다. 그래서 **conservative하게** 동작하며 5분 정도 기다립니다. 이게 너무 길어 보여도, 잘못 evict하면 더 큰 문제가 생깁니다.

서비스

Endpoints 컨트롤러

서비스 셀렉터와 매치되는 파드 IP를 Endpoints 리소스에 기록

Service에는 셀렉터(label selector)가 있지만, Service 리소스 자체는 어떤 파드를 가리키는지 모릅니다. 그 매핑을 **Endpoints** 리소스가 담고 있고, 이것을 만드는 게 Endpoints 컨트롤러입니다.

동작

1. Service 셀렉터와 매치되는 Pod들의 IP를 수집
 2. Ready 상태인 Pod만 추려서 Endpoints에 기록
 3. Pod가 변경되면 즉시 Endpoints 갱신
- kube-proxy는 Endpoints를 Watch해서 iptables 규칙을 갱신합니다.

최신 변화: 대규모 서비스(수천 엔드포인트)에서는 단일 Endpoints가 너무 커져 문제가 됩니다. K8s 1.21+에서는 **EndpointSlices**가 기본이 되었습니다.

보안

ServiceAccount 컨트롤러

네임스페이스마다 기본 SA 생성, 토큰 발급

- 네임스페이스마다 **default** ServiceAccount 자동 생성
- (1판 시절) SA마다 토큰 Secret 자동 발급
- Admission Controller가 파드에 SA 토큰을 자동 마운트

2판 변화: K8s 1.24+ 부터는 SA에 자동으로 영구 토큰을 만들지 않습니다. 대신 **TokenRequest API**로 단기 토큰을 발급 받아 projected volume으로 마운트합니다. 보안 강화입니다.

스토리지

PV 컨트롤러

PVC를 PV에 바인딩, 동적 프로비저닝 처리

두 가지 모드

- **정적 프로비저닝:** 관리자가 미리 PV를 만들어둠
→ 컨트롤러는 PVC에 맞는 PV를 찾아 binding
- **동적 프로비저닝:** StorageClass를 보고 자동으로 PV 생성 (예: AWS EBS, GCE PD)

매칭 기준

스토리지 크기, accessModes, StorageClass, selector 등을 매치합니다. 매치되는 PV가 없으면 PVC는 **Pending** 상태로 남습니다.

최신: CSI(Container Storage Interface) 드라이버가 표준입니다. in-tree 볼륨 플러그인들은 모두 CSI로 마이그레이션되었습니다.

2판 변화

Endpoints → EndpointSlices

K8s 1.21+ 부터 대규모 서비스의 성능을 위해 **EndpointSlices** 가 기본이 되었습니다. 1판의 단일 거대 Endpoints 리소스는 수천 개 엔드포인트에서 성능 문제가 있었는데, EndpointSlices는 여러 작은 슬라이스로 나눠 효율적입니다.

이 슬라이드의 용어 풀이

네임스페이스

여기서는 리눅스 커널의 격리 기능이에요. 프로세스마다 네트워크·PID 등을 따로 보이게 해서 컨테이너를 서로 격리해요. (K8s의 namespace 리소스와는 다른 개념!)

컨트롤러 매니저

여러 "컨트롤러"를 한데 모아 돌리는 프로세스예요. 각 컨트롤러는 "원하는 상태"와 "현재 상태"를 비교해 계속 맞춰주는 일을 해요.

Admission Control

인증·인가를 통과한 요청을 etcd에 저장하기 직전에 "정책 검사/자동 수정"하는 단계예요. 예: 리소스 한도 초과 막기, 기본값 채워주기 등.

Endpoints

Service가 트래픽을 보낼 "실제 파드 IP 목록"이에요. 파드가 늘거나 죽으면 이 목록이 갱신되고, kube-proxy가 그걸 보고 규칙을 다시 써요.

EndpointSlices

Endpoints의 개선판. 파드가 수천 개여도 작은 조각(슬라이스)으로 쪼개 관리해서, 하나 바뀔 때 전체가 아니라 해당 조각만 갱신해요.

iptables

리눅스 커널의 패킷 필터링/변환 규칙 시스템이에요. kube-proxy가 여기에 규칙을 넣어 Service IP로 온 트래픽을 실제 파드로 보내요.

kube-proxy

각 노드에서 Service(가상 IP)로 온 트래픽을 실제 파드로 전달되도록 네트워크 규칙(iptables 등)을 관리하는 컴포넌트예요.

Reconciliation Loop

"조정 루프"예요. 컨트롤러가 "내가 원하는 상태(예: 파드 3개)"와 "지금 실제 상태"를 비교해서, 다를 때마다 같아지도록 계속 고치는 무한 반복 동작이에요.

Secret

비밀번호·토큰·인증서 같은 민감한 값을 담는 리소스예요. ConfigMap과 비슷하지만 민감 정보용이에요.

ServiceAccount

사람이 아니라 "파드(앱)"가 API 서버에 접근할 때 쓰는 계정이에요. 파드 안에 토큰이 자동으로 들어가요.

Watch

API 서버가 "리소스가 바뀌면 알려줄게"라고 변경 사항을 실시간으로 밀어주는(push) 기능이에요. 계속 물어보는 폴링보다 효율적이에요.

11.1.7 Kubelet

워커 노드의 "주인공". 노드에서 일어나는 모든 일을 책임집니다.

🎯 **한 줄 결론:** kubelet은 각 노드의 **현장 반장**이에요. "내 노드에 배정된 파드"를 API 서버에서 받아, 컨테이너 런타임을 시켜 **실제로 컨테이너를 띄우고**, 잘 살아있는지 확인해서 보고해요.

🍔 **쉽게 말하면:** 공사 현장의 **현장소장**이에요. 본사(API 서버)에서 "이 동 지어"라는 지시서를 받아, 인부(컨테이너 런타임)에게 실제 작업을 시키고, 진행 상황을 본사에 보고하죠. 스케줄러가 "어디 지을지"를 정했다면, 실제로 짓는 건 kubelet이에요.

Kubelet의 책임

- ▶ **노드 등록:** API 서버에 Node 리소스로 자기 자신을 등록
- ▶ **파드 실행:** 할당된 파드를 컨테이너 런타임에 지시하여 실행
- ▶ **볼륨 마운트:** PV/CSI 볼륨을 컨테이너에 연결
- ▶ **프로브 실행:** liveness, readiness, startup probe 직접 수행
- ▶ **상태 보고:** 노드 상태와 파드 상태를 API 서버에 주기적으로 보고
- ▶ **Static Pod 관리:** 매니페스트 디렉토리의 정적 파드 실행

런타임

CRI: 컨테이너 런타임 인터페이스

kubelet과 런타임 간 [gRPC](#) 표준 인터페이스

왜 CRI가 필요했나?

초기 K8s는 Docker에 직접 결합되어 있었습니다. 다양한 런타임(rkt, containerd 등)을 지원하기 위해 **gRPC 기반 표준 인터페이스**를 만들었고, 이게 CRI입니다.

CRI 구조

- **RuntimeService**: Pod/컨테이너 생명주기 관리
- **ImageService**: 이미지 pull, list, 삭제

주요 런타임

- **containerd**: Docker가 내부적으로 쓰던 런타임이 독립. 가장 널리 쓰임.
- **CRI-O**: K8s 전용으로 설계된 가벼운 런타임
- **Docker**: dockershim을 통해 지원했으나 K8s 1.24에서 제거됨

특수

Static Pod

API 서버 없이도 동작 ([컨트롤 플레인](#) 부트스트랩에 사용)

Static Pod란?

kubelet이 **API 서버를 통하지 않고** 로컬 디렉토리 (`/etc/kubernetes/manifests`)의 매니페스트를 직접 읽어 실행하는 파드입니다.

왜 필요한가?

닭과 달걀 문제! API 서버 자체를 실행하려면 K8s가 필요한데, K8s가 동작하려면 API 서버가 필요합니다. 그래서 **API 서버, etcd, scheduler, controller-manager가 모두 static pod로 실행**됩니다. (kubeadm 클러스터의 표준)

특징

- kubelet이 직접 관리 → API 서버가 죽어도 동작
- API에서는 "mirror pod"로 보이지만, 거기서 삭제해도 안 죽음
- 실제 삭제는 매니페스트 파일을 지워야 함

프로브 (Probe) 실행

liveness/readiness/startup probe를 kubelet이 직접 수행

세 가지 프로브

- **livenessProbe**: 죽었나? → 실패 시 컨테이너 재시작
- **readinessProbe**: 트래픽 받을 준비 됐나? → 실패 시 Endpoints에서 제외
- **startupProbe**: (K8s 1.16+) 시작 중인가? → 통과 전까지 liveness 비활성

프로브 유형

- **httpGet**: 특정 경로에 HTTP GET
- **tcpSocket**: TCP 포트 연결만 확인
- **exec**: 컨테이너 내 명령 실행
- **grpc**: (K8s 1.24+) gRPC 헬스 체크

실수 주의: liveness가 너무 공격적이면 시작이 느린 앱이 계속 재시작 됩니다. startupProbe로 보호하세요.

2판 변화 (매우 중요!)

Docker 지원 제거 (dockershim deprecation)

1판은 Docker가 기본 런타임이라고 설명하지만, **K8s 1.20에서 deprecated, 1.24에서 dockershim이 완전히 제거**되었습니다. 현재는 containerd 또는 CRI-O를 사용해야 합니다. Docker로 빌드한 이미지는 OCI 표준이라 여전히 동작하며, 단지 노드의 런타임이 바뀐 것입니다.

이 슬라이드의 용어 풀이

스케줄러

"이 파드를 어느 노드에서 실행할까?"만 결정하는 컴포넌트예요. 파드를 직접 만들거나 실행하지는 않아요. 조건에 맞는 노드를 고르는 게 전부예요.

워커 노드

실제로 앱 컨테이너가 돌아가는 일꾼 서버예요. 컨트롤 플레인인 "이 파드를 띄워"라고 시키면 여기서 실행돼요.

인가

"그걸 할 권한 있어?"를 확인하는 단계예요. (영문: Authorization) 신원이 확인된 뒤, 그 사람이 해당 작업을 해도 되는지 RBAC 등으로 검사해요.

컨테이너 런타임

컨테이너를 실제로 실행하는 엔진이에요. kubelet이 "이 이미지 띄워"라고 하면 진짜로 띄우는 일꾼. 요즘은 containerd나 CRI-O를 써요.

컨트롤 플레인

클러스터의 "두뇌" 역할을 하는 영역이에요. 무엇을 어디서 실행할지 결정하는 관리 컴포넌트(API 서버·etcd·스케줄러·컨트롤러 매니저)가 모여 있어요. 예전엔 "마스터 노드"라고 불렀어요.

API 서버

모든 컴포넌트가 거쳐가는 중앙 창구예요. kubectl·스케줄러·kubelet 등은 서로 직접 대화하지 않고 전부 API 서버를 통해서만 통신해요. (영문: API Server)

containerd

가장 널리 쓰이는 컨테이너 런타임이에요. 원래 Docker 내부에서 쓰던 엔진이 독립한 것. Docker로 만든 이미지도 그대로 실행돼요.

CRI

Container Runtime Interface. kubelet과 컨테이너 런타임 사이의 "표준 약속"이에요. 이 규격만 맞추면 어떤 런타임이든 K8s에 끼울 수 있어요.

CRI-O

쿠버네티스 전용으로 가볍게 설계된 컨테이너 런타임이에요. OpenShift의 기본 런타임이에요.

dockershim

예전에 kubelet이 Docker를 호출하려고 두었던 중간 어댑터예요. K8s 1.24에서 제거됐고, 지금은 containerd/CRI-O를 직접 써요.

Endpoints

Service가 트래픽을 보낼 "실제 파드 IP 목록"이에요. 파드가 늘거나 죽으면 이 목록이 갱신되고, kube-proxy가 그걸 보고 규칙을 다시 써요.

etcd

클러스터의 모든 정보(어떤 파드가 있고 상태가 어떤지 등)를 저장하는 데이터베이스예요. 클러스터의 "공식 장부"라고 보면 돼요. 오직 API 서버만 직접 접근할 수 있어요.

gRPC

구글이 만든 빠른 통신 방식이에요. 일반 REST(JSON)보다 가볍고 효율적이라 etcd v3가 이걸 써요.

Kubelet

각 워커 노드의 주인공 에이전트예요. API 서버에서 "이 노드에 배정된 파드"를 받아, 컨테이너 런타임에게 시켜 실제로 컨테이너를 띄우고 상태를 보고해요.

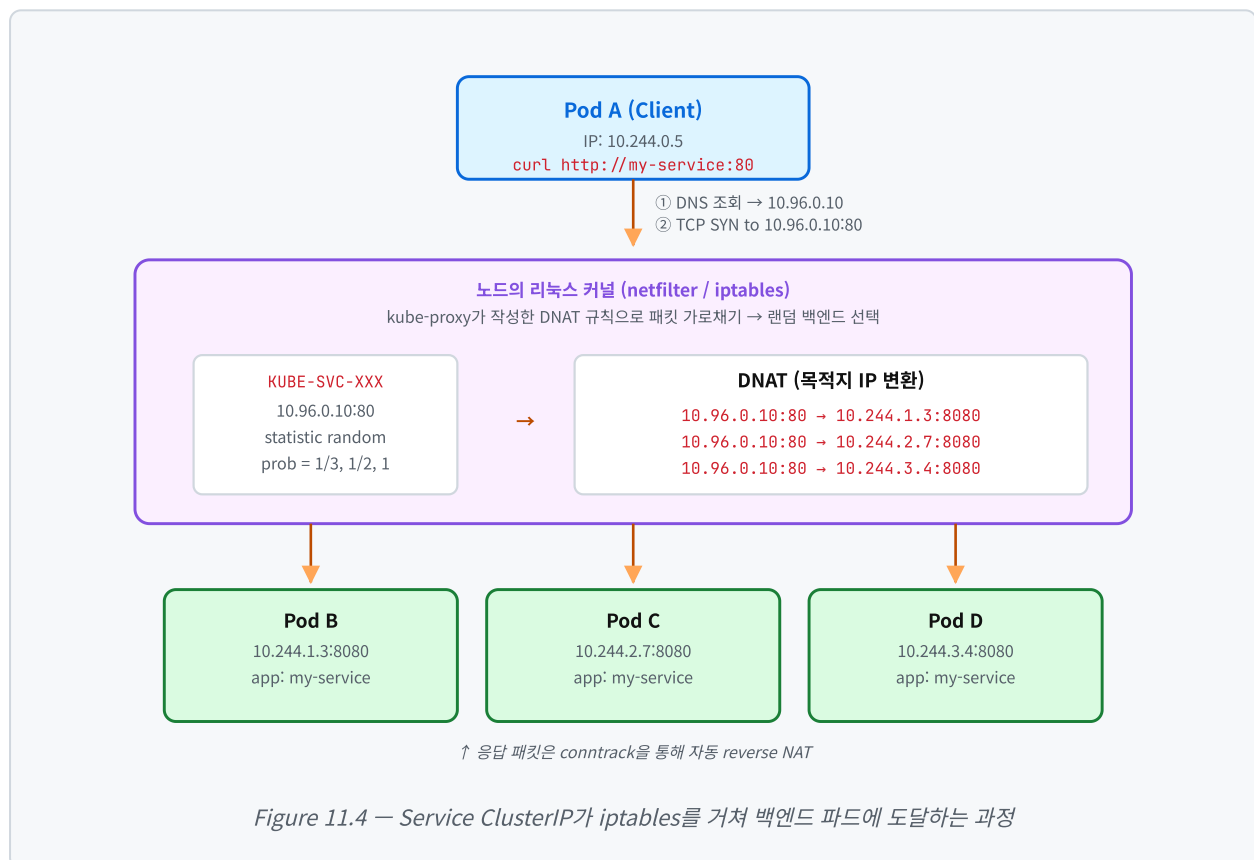
11.1.8 kube-proxy: 서비스 프록시

서비스(ClusterIP)로 향하는 트래픽을 실제 파드 IP로 전달합니다.

🎯 **한 줄 결론:** 앱은 안 바뀌는 **대표 주소(서비스 IP)**로 요청을 보내요. 그러면 **kube-proxy**가 미리 깔아둔 규칙(**iptables**)이 그 가짜 IP를 **실제 파드 IP로 바꿔서** 전달해줘요. kube-proxy 자신이 트래픽을 직접 나르진 않아요 — 규칙만 깔아둘 뿐이죠.

🍔 **쉽게 말하면:** 회사 **대표번호(1588)** 같은 거예요. 손님은 대표번호로 걸지만(서비스 IP), 교환기가 알아서 비어 있는 **상담원(파드)**에게 연결해줘요. 상담원이 바뀌어도 대표번호는 그대로니까 편하죠.

서비스 트래픽이 흐르는 길



userspace 모드

(1판) 초기 방식. kube-proxy 자체가 트래픽을 받아 전달 → 느림

kube-proxy 자신이 **userspace**에서 트래픽을 직접 받아 백엔드 파드로 전달했습니다.

흐름

1. iptables가 ClusterIP 트래픽을 kube-proxy로 라우팅
2. kube-proxy가 패킷을 받아 백엔드 선택
3. kube-proxy가 패킷을 다시 보냄

왜 느렸나?

매 패킷마다 **커널 ↔ 유저 공간 전환**이 일어나서 오버헤드가 큼니다. 그래서 iptables 모드로 진화했습니다.

iptables 모드

커널 iptables 규칙만 작성. 트래픽은 커널이 처리 → 빠름

핵심 아이디어

kube-proxy는 **iptables** 규칙을 작성만 하고, 실제 트래픽 처리는 **리눅스 커널**이 합니다. 패킷이 유저 공간을 거치지 않아 훨씬 빠릅니다.

규칙 작성

- Service 마다 KUBE-SVC-XXX 체인 생성
- Endpoint 마다 KUBE-SEP-YYY 체인 생성 (DNAT 규칙)
- SVC 체인은 statistic 모듈로 **랜덤 분산**

한계

서비스가 늘어나면 iptables 규칙도 늘어나고, 규칙 검색은 **O(n)**입니다. 수천 개의 서비스를 가진 클러스터에서는 패킷 처리가 느려질 수 있습니다.

IPVS 모드

대규모(수천 서비스) 환경에 최적화. 다양한 LB 알고리즘 지원

IPVS란?

리눅스 커널의 **IP Virtual Server**. L4 로드 밸런서를 커널 내부에서 구현한 것입니다. iptables보다 빠르고 다양한 알고리즘을 지원합니다.

장점

- 해시 테이블 기반 → **O(1)** 규칙 검색
- 다양한 LB 알고리즘: rr(round-robin), lc(least-connection), dh(destination-hashing), sh(source-hashing) 등
- 대규모(수천 서비스) 환경에서 안정적 성능

활성화

`kube-proxy --proxy-mode=ipvs`. 노드에 `ip_vs` 커널 모듈이 로드되어 있어야 합니다.

최신 트렌드: Cilium 같은 eBPF 기반 솔루션은 **kube-proxy 자체를 대체**합니다. 미래는 eBPF.

IPVS 모드 추천 & nftables 모드 등장

대규모 클러스터(수천 서비스)에서는 `iptables` 규칙 검색이 O(n) 이라 성능이 떨어집니다. **IPVS 모드**는 해시 테이블 기반(O(1))으로 빠르며 round-robin, least-conn 등 다양한 알고리즘을 지원합니다. 또한 K8s 1.29+ 에서는 **nftables 모드(beta)**도 등장했습니다.

이 슬라이드의 용어 풀이

ClusterIP

Service의 기본 유형. 클러스터 "내부에서만" 접근 가능한 가상 IP를 줘요. 실제 서버가 아니라 iptables 규칙으로 만들어진 가짜 IP예요.

conntrack

리눅스가 "이 연결은 아까 어디로 보냈지"를 기억하는 연결 추적 테이블이에요. 덕분에 응답 패킷이 자동으로 원래 길로 되돌아가요.

DNAT

Destination NAT. 패킷의 "목적지 IP"를 바꿔치기하는 거예요. Service의 가상 IP로 온 패킷의 목적지를 실제 파드 IP로 바꿔 전달할 때 써요.

eBPF

리눅스 커널 안에서 안전하게 작은 프로그램을 실행하는 기술이에요. Cilium이 이걸로 kube-proxy를 대체하고 빠른 네트워킹/보안을 제공해요.

iptables

리눅스 커널의 패킷 필터링/변환 규칙 시스템이에요. kube-proxy가 여기에 규칙을 넣어 Service IP로 온 트래픽을 실제 파드로 보내요.

IPVS

리눅스 커널의 로드밸런싱 기능이에요. iptables보다 규칙이 많아도 빨라서(O(1)) 서비스가 수천 개인 대규모 환경에 권장돼요.

kube-proxy

각 노드에서 Service(가상 IP)로 온 트래픽을 실제 파드로 전달되도록 네트워크 규칙(iptables 등)을 관리하는 컴포넌트예요.

11.1.9 쿠버네티스 애드온

코어가 아닌, 그러나 사실상 필수인 컴포넌트들. 보통 파드로 배포됩니다.

🔗 **한 줄 결론:** 쿠버네티스 본체에는 없지만 **실제 운영에선 거의 항상 깔게 되는 부가 기능들**이에요(DNS, 네트워크 플러그인, 모니터링 등). 특별한 게 아니라 대부분 그냥 **파드로 돌아가요**.

🍔 **쉽게 말하면:** 스마트폰의 "기본 탑재는 아니지만 다들 까는 앱" 같은 거예요. 카톡처럼요. 특히 **DNS(서비스 이름 찾기)**와 **CNI(파드 네트워크)**는 사실상 안 깔면 클러스터가 제대로 안 돌아가요.

필수

DNS 서버

서비스 이름 → [ClusterIP](#) 해석. 1판: KubeDNS, 최신: CoreDNS

왜 클러스터 DNS가 필요한가?

파드 IP는 자주 바뀌지만 Service의 ClusterIP는 안정적입니다. 하지만 그것도 매번 IP로 접근하긴 불편하죠. 그래서 DNS로 **my-service.default.svc.cluster.local** 같은 이름으로 접근할 수 있게 합니다.

1판: KubeDNS

3개 컨테이너(kubedns, dnsmasq, sidecar)로 구성된 복잡한 구조.

최신: CoreDNS (K8s 1.11+ 기본)

플러그인 기반의 단일 Go 바이너리. 가볍고 확장성 좋음. **Corefile**로 동작 커스터마이징 가능.

DNS 해석 규칙

- **my-svc** → 같은 namespace의 my-svc
- **my-svc.other-ns** → other-ns의 my-svc
- FQDN: **my-svc.other-ns.svc.cluster.local**

UI

Dashboard

웹 기반 클러스터 관리 UI

쿠버네티스 공식 웹 UI입니다. 파드, Deployment, Service 등을 시각적으로 관리할 수 있습니다.

실무에서는?

운영 환경에서는 **거의 사용하지 않습니다**. 보안 위험(과거 LB로 노출했다가 크립토 마이닝 당한 사례 다수), 그리고 더 강력한 도구들(Lens, k9s, Argo CD UI 등)이 많기 때문입니다.

대안

- **k9s**: 터미널 UI (개발자 인기 최고)
- **Lens**: 데스크톱 IDE
- **Headlamp**: 최신 웹 UI

Ingress Controller

L7 HTTP/HTTPS 라우팅. Nginx, Traefik, HAProxy 등

Ingress vs Service

- **Service:** L4 (TCP/UDP) 로드 밸런싱
- **Ingress:** L7 HTTP 라우팅 (URL 경로, 호스트명 기반)

구조

- **Ingress 리소스:** K8s YAML로 정의 (라우팅 규칙)
- **Ingress Controller:** 실제 트래픽을 처리하는 프록시 (Nginx, Traefik 등)

Ingress 리소스만 만들고 컨트롤러가 없으면 아무 것도 안 됩니다. 별도로 설치해야 합니다.

2판 트렌드: Ingress는 표현력 한계가 있어 vendor 별 annotation으로 확장하는 식이었습니다. 그래서 **Gateway API**(K8s 1.29+ GA)가 후속 표준으로 등장했습니다.

모니터링 (Heapster → [Metrics Server](#))

1판의 Heapster는 deprecated. 지금은 Metrics Server + [Prometheus](#)

Heapster (1판)

K8s 초기의 메트릭 수집기. cAdvisor로부터 데이터를 모아 백엔드(InfluxDB, Stackdriver 등)로 전송했습니다.

2018년: Heapster Deprecated

역할이 너무 광범위했고, 통합 모니터링 솔루션 (Prometheus 등)이 더 잘 처리하게 되었습니다.

현재 표준

- **Metrics Server:** HPA / **kubectl top** 용 가벼운 메트릭 (CPU/메모리만)
- **Prometheus + Grafana:** 본격적인 시계열 모니터링
- **kube-state-metrics:** 리소스 상태 메트릭
- **OpenTelemetry:** 트레이싱 / 로그 표준

CNI 플러그인

파드 네트워크 구현. Calico, Flannel, Cilium 등

CNI의 역할

kubelet이 파드를 만들 때, **네트워크 설정**(IP 할당, 인터페이스 생성 등)을 CNI 플러그인에 위임합니다.

주요 CNI 플러그인

- **Flannel**: 단순, VXLAN 오버레이. 학습용 좋음.
- **Calico**: BGP 라우팅, NetworkPolicy 강력. 가장 인기.
- **Cilium**: eBPF 기반, L7까지 정책. 최신 트렌드.
- **Weave**: 메시 네트워크, 암호화 옵션.
- **AWS VPC CNI**: EKS에서 ENI를 직접 할당 (오버레이 없음)

NetworkPolicy

파드 간 통신을 제한하는 K8s 리소스. CNI가 지원해야 동작합니다 (Flannel은 미지원).

Gateway API

(최신) Ingress의 후속 표준. 더 풍부한 트래픽 라우팅

왜 등장했나?

Ingress는 단순한 HTTP 라우팅만 표현 가능했고, 고급 기능(가중치 트래픽 분할, mTLS, 헤더 기반 라우팅 등)은 vendor annotation으로 처리해야 했습니다. 표준화 부재.

Gateway API 구조

- **GatewayClass**: 인프라 제공자 (인프라 팀이 관리)
- **Gateway**: 실제 진입점 (네트워크 팀이 관리)
- **HTTPRoute / TCPRoute / TLSRoute**: 라우팅 규칙 (앱 팀이 관리)

장점

- 역할 분리 (RBAC 친화적)
- 풍부한 표현력 (가중치, 헤더, 미러링 등)
- Vendor 중립적 표준
- 서비스 메시(Istio 등)와도 연동

Heapster 종말 → Metrics Server + Prometheus 시대

1판에 나오는 **Heapster**는 2018년에 완전히 deprecated 되었습니다. 현재는:

- **Metrics Server**: [HPA](#), `kubectl top` 용 리소스 메트릭만 제공 (가벼움)
- **Prometheus + Grafana**: 본격적인 시계열 모니터링은 별도로 구성
- **Gateway API**: Ingress의 한계를 극복한 차세대 표준 (1.0 GA, K8s 1.29+)

이 슬라이드의 용어 풀이

BGP

인터넷 라우터들이 "이 IP 대역은 내 쪽으로 와"라고 경로를 알리는 프로토콜이에요. Calico는 이걸 써서 캡슐화 없이 노드 간 파드 통신을 라우팅해요.

cAdvisor

kubelet에 내장된, 노드와 컨테이너의 리소스 사용량을 수집하는 도구예요.

ClusterIP

Service의 기본 유형. 클러스터 "내부에서만" 접근 가능한 가상 IP를 줘요. 실제 서버가 아니라 iptables 규칙으로 만들어진 가짜 IP예요.

CNI

Container Network Interface. kubelet이 파드를 만들 때 "IP 할당, 네트워크 연결"을 위임하는 플러그인 표준이에요. Calico·Cilium·Flannel 등이 구현체예요.

eBPF

리눅스 커널 안에서 안전하게 작은 프로그램을 실행하는 기술이에요. Cilium이 이걸로 kube-proxy를 대체하고 빠른 네트워킹/보안을 제공해요.

Gateway API

Ingress의 후속 표준이에요(K8s 1.29+ GA). 가중치 분할·헤더 라우팅 등 표현력이 풍부하고, 인프라/네트워크/앱 팀의 역할을 나눠 관리해요.

HPA

Horizontal Pod Autoscaler. CPU/메모리 사용량 등을 보고 파드 개수를 자동으로 늘리거나 줄여주는 기능이에요.

Ingress

클러스터 밖에서 들어오는 HTTP(L7) 트래픽을 URL 경로·호스트명 기준으로 여러 Service에 나눠 보내는 라우팅 규칙이에요.

Metrics Server

CPU/메모리 사용량을 가볍게 모아주는 컴포넌트예요. kubectl top과 HPA가 이걸 써요. 장기 저장은 안 해요.

Prometheus

시계열(시간별) 메트릭을 저장하고 알림·그래프를 제공하는 모니터링 도구예요. Grafana와 함께 많이 써요.

RBAC

Role-Based Access Control. "역할(Role)"에 권한을 묶고, 사용자/서비스에 역할을 부여해 권한을 관리하는 방식이에요. K8s의 표준 인가 방법이에요.

VXLAN

파드 패킷을 UDP로 한 번 감싸서 노드 사이를 건너게 하는 오버레이 기술이에요. Flannel의 기본 방식이에요.

11.2 컨트롤러들의 협력

Deployment YAML 하나를 적용했을 때, 내부에서 무슨 일이 벌어지는가?

🎯 **한 줄 결론:** `kubectl apply` 한 번이 즉시 실행되는 게 아니에요! **Deployment → ReplicaSet → Pod → 스케줄러 → kubelet** 순서로, 각자 Watch(알림)만 보고 자기 차례 일을 이어받는 릴레이예요. 아무도 남에게 직접 명령하지 않아요 — 모두 API 서버만 보고 움직이죠.

🍔 **쉽게 말하면:** 도미노예요. 전체를 지휘하는 감독이 없어도, 각 조각이 "내 차례 신호(Watch)"를 보고 다음으로 넘겨요. 그래서 컴포넌트 하나가 잠깐 멈춰도 전체가 무너지지 않고, 다시 살아나면 멈춘 지점부터 이어가요.

파드가 생성되기까지의 여정 (Deployment 기준)

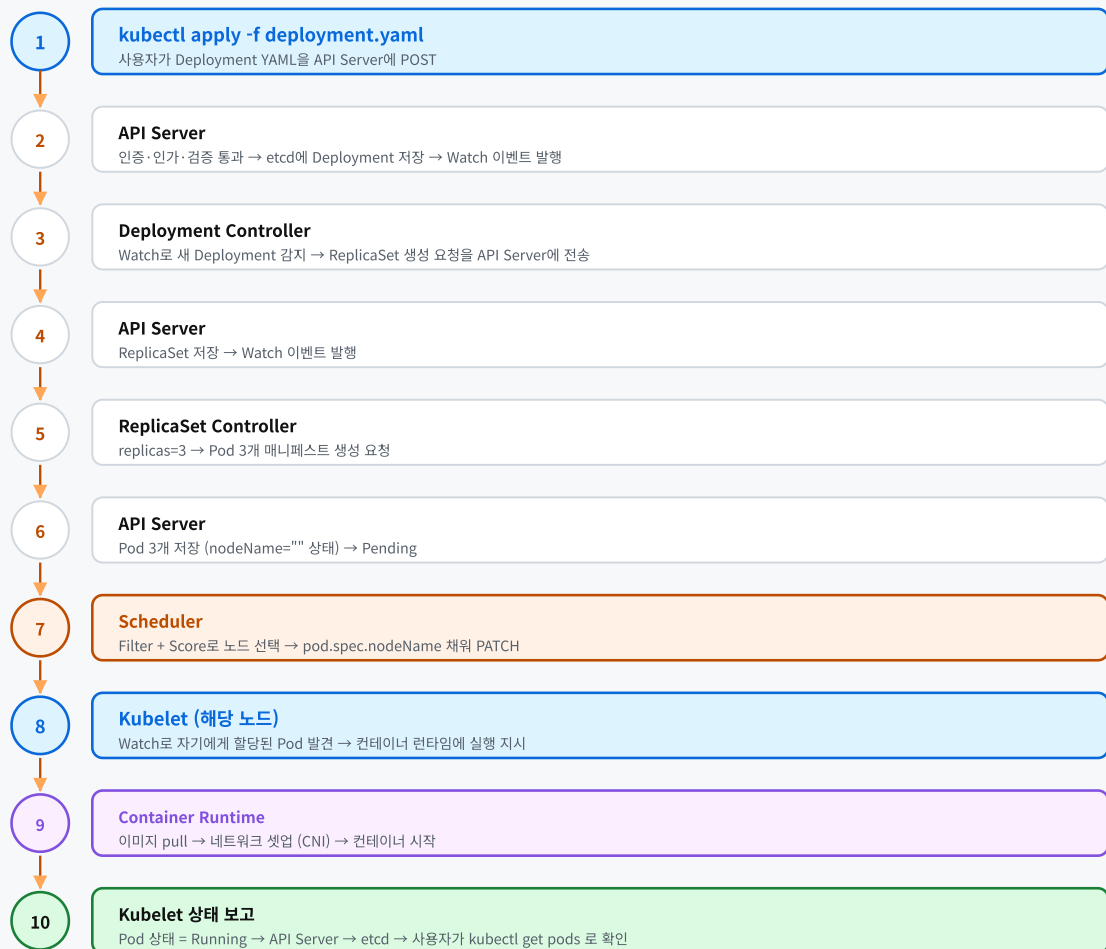


Figure 11.5 — `kubectl apply` 하나가 10개의 단계를 거쳐 Pod로 실행되기까지

관찰

이벤트로 흐름 관찰하기

`kubectl get events`로 이 흐름을 시각화

실습

```
$ kubectl run nginx --image=nginx
$ kubectl get events --sort-by=.metadata.creationTimestamp
```

출력 예시:

Scheduled	Successfully assigned def
Pulling	Pulling image "nginx"
Pulled	Successfully pulled image
Created	Created container nginx
Started	Started container nginx

각 이벤트가 다른 컴포넌트에서 발생했음을 확인할 수 있습니다:

- **Scheduled** ← Scheduler
- **Pulling/Pulled** ← Kubelet (런타임 호출)
- **Created/Started** ← Kubelet

설계

왜 이렇게 분리되어 있는가?

각 컨트롤러는 단일 책임 · API 서버만이 진실의 원천

설계 원칙

- **단일 책임**: 각 컨트롤러는 한 가지만 한다
- **느슨한 결합**: 컨트롤러는 서로의 존재를 모름. API 서버만 안다
- **레벨 트리거**: "현재 상태"를 보고 조정. "이벤트"에 의존 X (이벤트는 놓칠 수 있음)
- **幂등성**: 같은 동작을 여러 번 해도 결과 같음 (안전한 재시도)

장점

- 컨트롤러 하나가 죽어도 다른 건 영향 X
- 새 컨트롤러를 추가하기 쉬움 (Operator)
- 네트워크 분할/일시 장애에 견고함

💡 **Operator 패턴**의 기반이 바로 이 설계입니다. CRD + 커스텀 컨트롤러로 K8s를 무한 확장할 수 있습니다.

💡 **발표 포인트**: "kubectl apply"가 즉시 실행되지 않는 이유! 각 컴포넌트는 **독립적으로** Watch만 하면서 자기 일을 합니다. 이게 K8s의 핵심 설계 철학이고, 컴포넌트 하나가 죽어도 클러스터가 견고한 이유입니다.

📖 이 슬라이드의 용어 풀이

레벨 트리거

"지금 상태가 어떤가"를 보고 동작하는 방식이에요. "방금 무슨 이벤트가 일어났나"(엣지 트리거)에 의존하지 않아서, 이벤트를 한 번 놓쳐도 다음 번에 현재 상태를 보고 복구할 수 있어 견고해요.

멱등성

같은 동작을 여러 번 반복해도 결과가 똑같은 성질이에요. 예: "파드 3개를 만들어"를 두 번 실행해도 6개가 되지 않고 3개로 유지돼요. 그래서 안전하게 재시도할 수 있어요.

스케줄러

"이 파드를 어느 노드에서 실행할까?"만 결정하는 컴포넌트예요. 파드를 직접 만들거나 실행하지는 않아요. 조건에 맞는 노드를 고르는 게 전부예요.

인가

"그걸 할 권한 있어?"를 확인하는 단계예요. (영문: Authorization) 신원이 확인된 뒤, 그 사람이 해당 작업을 해도 되는지 RBAC 등으로 검사해요.

인증

"너 누구야?"를 확인하는 단계예요. (영문: Authentication) 인증서·토큰·ServiceAccount 등으로 요청자의 신원을 확인해요.

컨테이너 런타임

컨테이너를 실제로 실행하는 엔진이에요. kubelet이 "이 이미지 띄워"라고 하면 진짜로 띄우는 일꾼. 요즘은 containerd나 CRI-O를 써요.

API 서버

모든 컴포넌트가 거쳐가는 중앙 창구예요. kubectl·스케줄러·kubelet 등은 서로 직접 대화하지 않고 전부 API 서버를 통해서만 통신해요. (영문: API Server)

API Server

모든 컴포넌트가 거쳐가는 중앙 창구예요. 서로 직접 대화하지 않고 전부 여기를 통해서만 통신해요. etcd에 접근하는 유일한 통로이기도 해요.

CNI

Container Network Interface. kubelet이 파드를 만들 때 "IP 할당, 네트워크 연결"을 위임하는 플러그인 표준이에요. Calico·Cilium·Flannel 등이 구현체예요.

Container Runtime

컨테이너를 실제로 실행하는 엔진이에요. kubelet의 지시를 받아 진짜로 컨테이너를 띄워요. 요즘은 containerd나 CRI-O를 써요.

CRD

Custom Resource Definition. K8s에 "내가 만든 새로운 리소스 종류"를 추가하는 기능이에요. Operator 패턴의 기반이에요.

etcd

클러스터의 모든 정보(어떤 파드가 있고 상태가 어떤지 등)를 저장하는 데이터베이스예요. 클러스터의 "공식 장부"라고 보면 돼요. 오직 API 서버만 직접 접근할 수 있어요.

Kubelet

각 워커 노드의 주인공 에이전트예요. API 서버에서 "이 노드에 배정된 파드"를 받아, 컨테이너 런타임에게 시켜 실제로 컨테이너를 띄우고 상태를 보고해요.

Operator

CRD + 커스텀 컨트롤러로, 사람이 하던 운영 작업(DB 백업·복구 등)을 자동화하는 패턴이에요. K8s를 무한 확장하는 방법이에요.

Scheduler

"이 파드를 어느 노드에서 실행할까?"만 결정해요. 파드를 직접 만들거나 실행하지는 않고, 조건에 맞는 노드를 골라주는 역할이에요.

Watch

API 서버가 "리소스가 바뀌면 알려줄게"라고 변경 사항을 실시간으로 밀어주는(push) 기능이에요. 계속 물어보는 폴링보다 효율적이에요.

11.3 파드의 진짜 내부 구조 (숨은 pause 컨테이너)

파드 안에는 우리 컨테이너만 있는 게 아닙니다. 숨겨진 컨테이너가 있습니다.

이 절의 결론 한 줄: 내가 만든 적 없는 **pause** 컨테이너가 파드마다 몰래 하나 더 떠서 **네트워크·IP 공간을 만들어 붙잡고** 있고, 내 앱 컨테이너들은 그 공간을 **같이 쓴다(공유)**. 그래서 같은 파드 안 컨테이너끼리 **localhost**로 통신하고, 앱이 재시작돼도 파드 IP가 그대로 유지된다.

① 우리가 아는 파드 vs ② 실제로 뜨는 파드

```
# 내가 YAML에 적은 것
Pod
└─ 📦 my-app (내 앱 컨테이너)
```

"파드 = 내 컨테이너 담는 상자"로 알고 있죠.

```
# 노드에서 실제로 뜨는 것
Pod
├─ 🔒 pause ← 숨은 컨테이너!
└─ 📦 my-app (내 앱 컨테이너)
```

내가 안 만든 **pause**가 항상 하나 더 떠 있어요.

왜 굳이 숨은 컨테이너를 두는가? (핵심 문제)

같은 파드 안 컨테이너들은 **같은 IP를 공유**하고 **localhost**로 통신해야 합니다. 그러려면 누군가 그 "공용 네트워크 공간(네임스페이스)"을 만들어 들고 있어야 하는데 —

- ▶ **만약 내 앱 컨테이너가 그 공간의 주인이라면?** → 앱이 죽었다 재시작될 때 공간도 같이 사라져서 **파드 IP가 매번 바뀌어 버립니다.** 🤖
- ▶ **그래서 해결책:** 절대 안 죽고 아무 일도 안 하는 **pause**를 두고, 그 공간을 **pause가 붙잡게** 합니다. 앱들은 거기에 얹혀살 뿐이죠.
- ▶ 덕분에 앱 컨테이너가 몇 번을 재시작해도 **pause는 그대로 → 파드 IP·네트워크가 유지**됩니다.

🏠 **한 줄 비유:** **pause = 방을 계약하고 안 나가는 집주인**, **앱 컨테이너 = 같이 사는 룸메이트**. 룸메이트(앱)가 잠깐 나갔다 와도(재시작) 방(IP)은 그대로예요. 집주인(pause)이 계약을 계속 들고 있으니까요.

👉 이제 아래 그림을 보면, pause가 위에서 공간을 들고 있고 app-sidecar가 그걸 공유하는 구조가 보일 거예요.



📌 정리: pause가 하는 일 (한눈에)

- ▶ **공간을 만든다** — 파드의 네트워크/IPC 네임스페이스를 처음 생성
- ▶ **공간을 붙잡는다** — 안 죽고 계속 들고 있어 IP·네트워크 유지
- ▶ **공간을 공유시킨다** — 앱·sidecar가 같은 IP, **localhost** 통신 가능
- ▶ **거의 공짜다** — 무한 sleep만 하는 700KB짜리라 자원을 안 씀

더 깊이 보고 싶다면 아래 카드를 클릭하세요. (밑줄 친 용어도 클릭하면 쉬운 설명이 떠요)

상세

pause 컨테이너의 정체

몇 줄짜리 C 코드와 무한 sleep · 이미지 700KB

pause 컨테이너가 하는 일

거의 아무것도 안 합니다. C로 작성된 몇 줄짜리 프로그램이 무한 sleep 하는 게 전부입니다.

```
// pause.c (간략화)
int main() {
    signal(SIGINT, sigdown);
    signal(SIGTERM, sigdown);
    signal(SIGCHLD, sigreap); // 좀비 r
    for (;;) pause(); // 무한 대기
}
```

왜 필요한가?

- 파드의 네트워크/IPC 네임스페이스를 처음 생성하고 보류
- 다른 컨테이너들은 이 네임스페이스를 공유 (--net=container:pause)
- 앱 컨테이너가 죽고 재시작되어도 pause는 안 죽음 → 네임스페이스 보존
- 좀비 프로세스 reaping (PID 1 역할)

실제 보기

`docker ps` 또는 `crictl ps`에서 pod 당 `k8s_POD_*` 컨테이너를 볼 수 있습니다.

리눅스

리눅스 네임스페이스

Network, IPC, PID, Mount, UTS, User 등 6가지

네임스페이스 종류

- **Network**: 네트워크 인터페이스, 라우팅 테이블, iptables
- **IPC**: SystemV IPC, POSIX 메시지 큐
- **PID**: 프로세스 ID 공간
- **Mount**: 마운트 포인트
- **UTS**: 호스트네임
- **User**: UID/GID 매핑
- **cgroup**: cgroup 루트

파드에서의 공유

같은 파드의 컨테이너들은 **Network, IPC, UTS**를 공유합니다. 그래서 localhost로 통신할 수 있고, 호스트네임이 같습니다.

Mount와 PID는 기본적으로 분리됩니다 (단, `shareProcessNamespace: true`로 PID 공유 가능).

이 슬라이드의 용어 풀이

네임스페이스

여기서는 리눅스 커널의 격리 기능이에요. 프로세스마다 네트워크·PID 등을 따로 보이게 해서 컨테이너를 서로 격리해요. (K8s의 namespace 리소스와는 다른 개념!)

iptables

리눅스 커널의 패킷 필터링/변환 규칙 시스템이에요. kube-proxy가 여기에 규칙을 넣어 Service IP로 온 트래픽을 실제 파드로 보내요.

pause 컨테이너

파드마다 몰래 하나씩 뜨는 거의 아무것도 안 하는 컨테이너예요. 파드의 네트워크/IPC 공간을 처음 만들어 "붙잡고" 있어서, 앱 컨테이너가 재시작돼도 파드 IP가 유지돼요.

11.4 파드 간 네트워킹

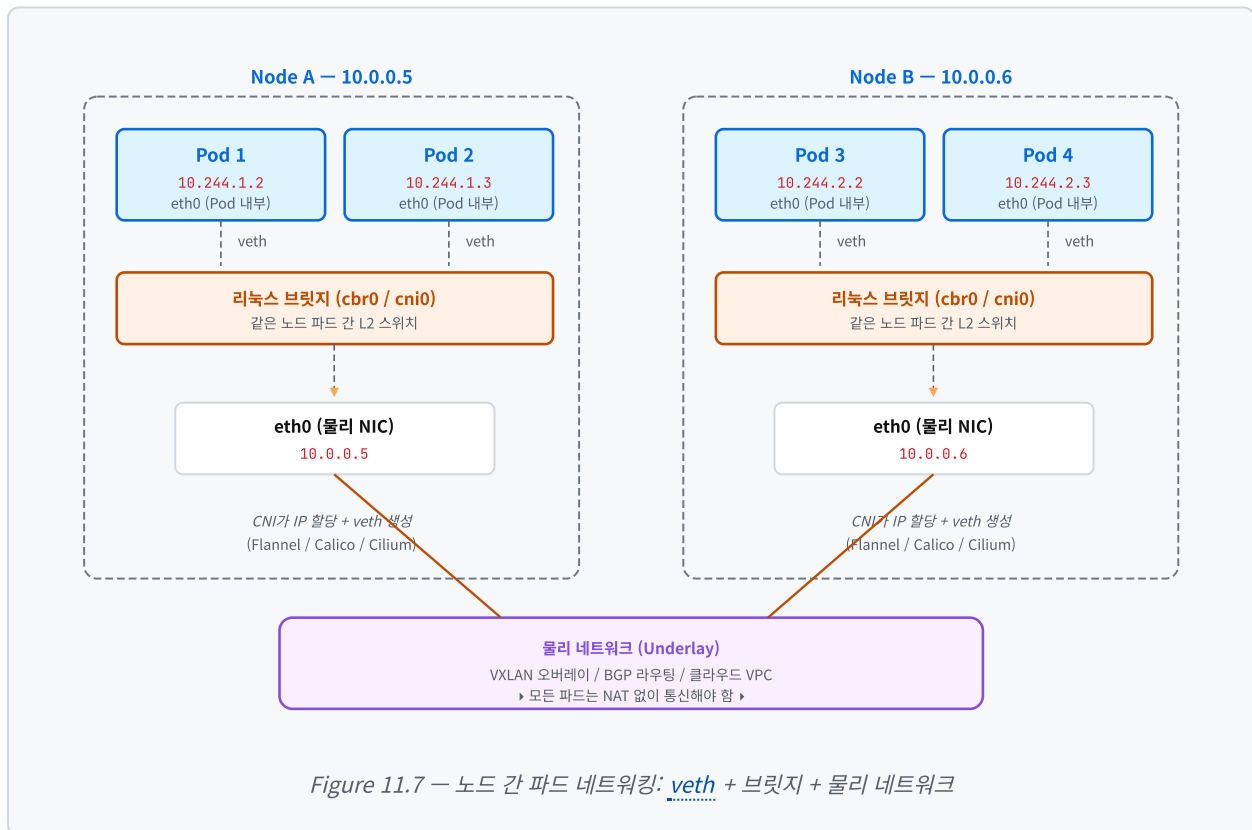
쿠버네티스 네트워크 모델은 단순합니다: 모든 파드가 NAT 없이 통신할 수 있어야 합니다.

🎯 **한 줄 결론:** 쿠버네티스의 약속은 "모든 파드는 고유 IP를 갖고, 주소 변환(NAT) 없이 서로 직접 통신한다"예요. 노드가 달라도 마치 같은 네트워크에 있는 것처럼요. 이 약속을 실제로 구현해주는 게 **CNI 플러그인**이에요.

🍔 **쉽게 말하면:** 거대한 아파트 단지 같아요. 모든 집(파드)이 고유 호수(IP)를 갖고 단지 내선전화로 바로 연결되죠. 같은 동(같은 노드) 안에서는 복도(브릿지)로, 다른 동(다른 노드)으로는 단지 도로(물리 네트워크)를 거쳐 연결돼요.

쿠버네티스 네트워크 모델의 3대 규칙

- ▶ 모든 파드는 다른 모든 파드와 **NAT 없이** 통신할 수 있어야 한다
- ▶ 모든 노드는 모든 파드와 **NAT 없이** 통신할 수 있어야 한다
- ▶ 파드가 자신을 보는 IP와 **다른 파드가 그 파드를 보는 IP가 같아야** 한다



노드 내부

같은 노드의 파드 간 통신

리눅스 브릿지 + veth 페어로 연결

구조

- 각 파드는 자기 네트워크 네임스페이스를 가짐
- 파드 내부에 **eth0** (veth pair의 한쪽)
- 호스트 쪽 veth는 **cbr0** 또는 **cni0** 브릿지에 연결
- 브릿지가 같은 노드의 파드 간 L2 스위치 역할

동작

Pod A → eth0(파드) → veth(호스트) → 브릿지
→ veth(호스트) → eth0(파드) → Pod B
이렇게 **리눅스 커널만으로** 같은 노드의 파드들이 서로 통신합니다.

노드 간

다른 노드의 파드 간 통신

[오버레이 네트워크 \(VXLAN\)](#) 또는 [BGP](#) 라우팅

방식 1: 오버레이 네트워크 (VXLAN)

파드 패킷을 UDP로 캡슐화해서 노드 간 전송.
Flannel의 기본 방식.

- 장점: 물리 네트워크 변경 불필요
- 단점: 캡슐화 오버헤드, MTU 이슈

방식 2: BGP 라우팅

각 노드를 라우터로 만들어, 노드끼리 파드 CIDR을 광고. Calico의 기본 방식.

- 장점: 캡슐화 없음 → 빠름
- 단점: 물리 네트워크가 BGP 지원해야 함 (또는 IPIP 풀백)

방식 3: 클라우드 ENI

AWS VPC CNI는 파드에 VPC IP를 직접 할당. ENI를 노드에 붙임. 오버레이 없음.

표준

CNI (Container Network Interface)

kubelet이 호출하는 표준 네트워크 플러그인 인터페이스

CNI 인터페이스

CNI는 단순합니다. kubelet이 파드 생성/삭제 시 다음을 호출합니다:

- **ADD**: 네트워크 인터페이스 생성, IP 할당
- **DEL**: 인터페이스 정리, IP 반환
- **CHECK**: 네트워크가 여전히 정상인가 확인

설정 파일

/etc/cni/net.d/에 JSON 설정.

/opt/cni/bin/에 플러그인 바이너리.

플러그인 체인

여러 플러그인을 연결할 수 있습니다. 예: **flannel**로 IP 할당 → **portmap**으로 hostPort → **bandwidth**로 대역폭 제한.

eBPF 기반 네트워킹의 부상 (Cilium)

1판은 Flannel/Calico/Weave 중심으로 설명하지만, 최신 트렌드는 **Cilium (eBPF 기반)**입니다. eBPF를 사용하면 [iptables](#) 없이 커널에서 직접 패킷 처리가 가능해, [kube-proxy](#)를 대체할 수 있고 L7 정책, 옴서버 빌리티(Hubble)까지 제공합니다.

이 슬라이드의 용어 풀이

네임스페이스

여기서는 리눅스 커널의 격리 기능이에요. 프로세스마다 네트워크·PID 등을 따로 보이게 해서 컨테이너를 서로 격리해요. (K8s의 namespace 리소스와는 다른 개념!)

브릿지

같은 노드 안 파드들을 연결하는 가상 스위치예요. (cbr0/cni0 등) 한 노드 안의 파드끼리는 이 브릿지를 통해 서로 통신해요.

오버레이 네트워크

실제 물리 네트워크 위에 "가상 네트워크"를 한 겹 더 얹는 방식이에요. 파드 패킷을 한 번 더 포장(캡슐화)해 노드 간에 전달해요. (예: VXLAN)

인가

"그걸 할 권한 있어?"를 확인하는 단계예요. (영문: Authorization) 신원이 확인된 뒤, 그 사람이 해당 작업을 해도 되는지 RBAC 등으로 검사해요.

BGP

인터넷 라우터들이 "이 IP 대역은 내 쪽으로 와"라고 경로를 알리는 프로토콜이에요. Calico는 이걸 써서 캡슐화 없이 노드 간 파드 통신을 라우팅해요.

CIDR

IP 주소 대역을 표기하는 방식이에요. 예: 10.244.0.0/16 은 "10.244.x.x 전부". 파드/서비스에 줄 IP 범위를 정할 때 써요.

CNI

Container Network Interface. kubelet이 파드를 만들 때 "IP 할당, 네트워크 연결"을 위임하는 플러그인 표준이에요. Calico·Cilium·Flannel 등이 구현체예요.

eBPF

리눅스 커널 안에서 안전하게 작은 프로그램을 실행하는 기술이에요. Cilium이 이걸로 kube-proxy를 대체하고 빠른 네트워킹/보안을 제공해요.

iptables

리눅스 커널의 패킷 필터링/변환 규칙 시스템이에요. kube-proxy가 여기에 규칙을 넣어 Service IP로 온 트래픽을 실제 파드로 보내요.

kube-proxy

각 노드에서 Service(가상 IP)로 온 트래픽을 실제 파드로 전달되도록 네트워크 규칙(iptables 등)을 관리하는 컴포넌트예요.

MTU

한 번에 보낼 수 있는 패킷 최대 크기예요. 오버레이는 포장 때문에 헤더가 늘어나 MTU를 잘 맞춰야 통신이 깨지지 않아요.

veth

virtual ethernet의 줄임말. 양 끝이 한 쌍인 가상 랜선이에요. 한쪽은 파드 안 eth0, 다른 쪽은 노드의 브릿지에 꽂아 파드를 네트워크에 연결해요.

VXLAN

파드 패킷을 UDP로 한 번 감싸서 노드 사이를 건너게 하는 오버레이 기술이에요. Flannel의 기본 방식이에요.

11.5 서비스(Service)가 동작하는 원리

ClusterIP는 사실 어떤 인터페이스에도 바인딩되지 않은 "가상 IP"입니다. 어떻게 동작할까요?

🔴 **한 줄 결론:** 파드는 자주 죽고 다시 뜰 때 IP가 바뀌어요. 그래서 **안 바뀌는 대표 주소(ClusterIP)**가 필요하죠. 이 IP는 어떤 장비에도 없는 "**환상의 IP**"고, kube-proxy가 깔아둔 iptables 규칙이 이걸 실제 파드로 연결해줘요.

🍌 **쉽게 말하면:** 회사 **대표번호/사서함**이에요. 직원(파드)이 바뀌어도 대표번호(서비스 IP)는 그대로죠. 그래서 **ping**(그 자리에 사람이 실제로 있나 확인)은 안 되지만, 전화 연결(TCP 통신)은 잘 돼요 — 규칙이 현재 직원에게 돌려주니까요.

iptables 모드의 마법

```
# Service 정의
Service: my-service (ClusterIP: 10.96.0.10:80)
Endpoints: 10.244.1.5:8080, 10.244.2.7:8080, 10.244.3.4:8080

# kube-proxy가 노드에 작성하는 iptables 규칙 (개념적)
-A KUBE-SERVICES -d 10.96.0.10/32 -p tcp --dport 80 \
  -j KUBE-SVC-XXXX

-A KUBE-SVC-XXXX -m statistic --mode random --probability 0.33333 \
  -j KUBE-SEP-AAAA # → 10.244.1.5:8080
-A KUBE-SVC-XXXX -m statistic --mode random --probability 0.50000 \
  -j KUBE-SEP-BBBB # → 10.244.2.7:8080
-A KUBE-SVC-XXXX -j KUBE-SEP-CCCC # → 10.244.3.4:8080

-A KUBE-SEP-AAAA -p tcp -j DNAT --to-destination 10.244.1.5:8080
```

흐름

요청이 흐르는 경로

DNS → ClusterIP → iptables [DNAT](#) → 실제 파드

전체 경로 (앱이 my-service:80 호출)

1. 앱이 my-
`service.default.svc.cluster.local`
DNS 조회
2. CoreDNS가 ClusterIP(예: 10.96.0.10) 반환
3. 앱이 10.96.0.10:80으로 TCP SYN 전송
4. 커널의 netfilter/iptables가 패킷을 가로챈
5. KUBE-SERVICES 체인 → KUBE-SVC-XXX 체인으로 점프
6. statistic 모듈로 랜덤 KUBE-SEP-YYY 선택
7. DNAT으로 목적지 IP를 실제 파드 IP(10.244.1.5)로 변경
8. conntrack 테이블에 기록 (응답 패킷의 SNAT용)
9. 패킷이 파드로 전달됨

응답 패킷은 conntrack을 통해 자동으로 reverse NAT 처리됩니다. 그래서 파드는 ClusterIP를 보지 못합니다.

동기화

[Endpoints](#)가 업데이트되는 흐름

파드 변경 → Endpoint Controller → kube-proxy → iptables 갱신

1. 파드가 Ready 상태가 됨 (또는 Terminating)
2. kubelet이 파드 상태를 API 서버에 보고
3. Endpoints 컨트롤러가 Watch → Endpoints 리소스 업데이트
4. 모든 노드의 kube-proxy가 Endpoints 변경을 Watch
5. 각 노드의 kube-proxy가 iptables 규칙 재작성
6. 이제 새 파드로 트래픽이 분산됨

지연(latency) 발생 지점

파드 변경부터 iptables 갱신까지 보통 수백ms ~ 수 초. 그래서 **graceful shutdown**(SIGTERM 받고 잠시 트래픽 받기) 패턴이 필요합니다.

유형

서비스 유형들의 구현 차이

ClusterIP / [NodePort](#) / [LoadBalancer](#) / ExternalName

ClusterIP (기본)

클러스터 내부에서만 접근 가능. 위에서 설명한 iptables DNAT으로 구현.

NodePort

모든 노드의 특정 포트(30000~32767)를 외부에 노출. 추가 iptables 규칙으로 노드 포트 → ClusterIP 흐름.

LoadBalancer

NodePort + 클라우드 LB. cloud-controller-manager가 클라우드 API로 LB를 만들고 노드를 백엔드로 등록.

ExternalName

iptables 규칙 없음! CoreDNS에 CNAME 레코드만 생성하는 특수 타입. 예: `db.example.com`으로 매핑.

Headless (clusterIP: None)

ClusterIP 할당 안 함. DNS가 직접 파드 IP들을 반환 (A 레코드 여러 개). StatefulSet에서 자주 사용.

💡 **발표 포인트:** ClusterIP는 어떤 네트워크 인터페이스에도 존재하지 않는 "**환상의 IP**"입니다. ping은 안 되지만 TCP/UDP는 됩니다 (iptables가 DNAT 해주니까). 이것 알면 서비스 디버깅이 훨씬 쉬워집니다.

이 슬라이드의 용어 풀이

API 서버

모든 컴포넌트가 거쳐가는 중앙 창구예요. kubectl·스케줄러·kubelet 등은 서로 직접 대화하지 않고 전부 API 서버를 통해서만 통신해요. (영문: API Server)

ClusterIP

Service의 기본 유형. 클러스터 "내부에서만" 접근 가능한 가상 IP를 줘요. 실제 서버가 아니라 iptables 규칙으로 만들어진 가짜 IP예요.

conntrack

리눅스가 "이 연결은 아까 어디로 보냈지"를 기억하는 연결 추적 테이블이에요. 덕분에 응답 패킷이 자동으로 원래 길로 되돌아가요.

DNAT

Destination NAT. 패킷의 "목적지 IP"를 바꿔치기하는 거예요. Service의 가상 IP로 온 패킷의 목적지를 실제 파드 IP로 바꿔 전달할 때 써요.

Endpoints

Service가 트래픽을 보낼 "실제 파드 IP 목록"이에요. 파드가 늘거나 죽으면 이 목록이 갱신되고, kube-proxy가 그걸 보고 규칙을 다시 써요.

iptables

리눅스 커널의 패킷 필터링/변환 규칙 시스템이에요. kube-proxy가 여기에 규칙을 넣어 Service IP로 온 트래픽을 실제 파드로 보내요.

kube-proxy

각 노드에서 Service(가상 IP)로 온 트래픽을 실제 파드로 전달되도록 네트워크 규칙(iptables 등)을 관리하는 컴포넌트예요.

LoadBalancer

클라우드의 로드밸런서를 자동으로 만들어 외부 트래픽을 받는 Service 유형이에요. NodePort 위에 클라우드 LB가 얹힌 형태예요.

NodePort

모든 노드의 특정 포트(30000~32767)를 열어 클러스터 "밖에서도" 접근하게 하는 Service 유형이에요.

Watch

API 서버가 "리소스가 바뀌면 알려줄게"라고 변경 사항을 실시간으로 밀어주는(push) 기능이에요. 계속 물어보는 폴링보다 효율적이에요.

11.6 고가용성 클러스터 운영

애플리케이션 HA와 컨트롤 플레인 HA, 두 가지를 모두 고려해야 합니다.

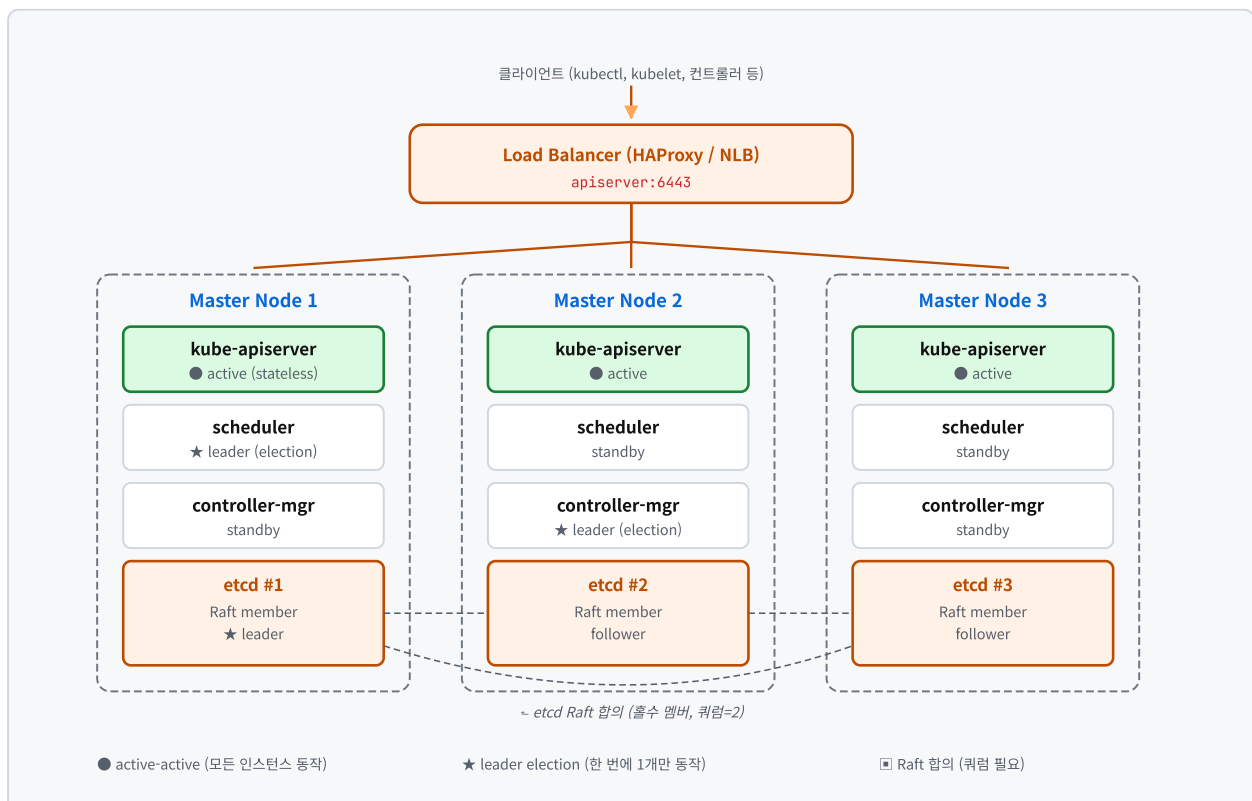
🎯 **한 줄 결론:** HA(고가용성) = "하나 죽어도 안 멈춤"이에요. 내 앱도, 쿠버네티스의 두뇌(컨트롤 플레인)도 여러 개를 뒀어 해요. API 서버는 전부 동시에 일하고(stateless), 스케줄러·컨트롤러는 **한 명만 일하고 나머지는 대기(리더 선출)**, etcd는 홀수로 두어 과반 합의해요.

🍔 **쉽게 말하면:** 비행기 엔진 여러 개를 다는 것과 같아요. 엔진 하나가 꺼져도 나머지로 계속 날 수 있죠. 단, 조종간은 **한 명만 잡아야** 충돌이 안 나요(스케줄러·컨트롤러의 리더 선출). 여러 명이 동시에 조종하면 엉키니까요.

1) 애플리케이션의 가용성

- ▶ **Deployment + 다중 레플리카:** 한 파드가 죽어도 다른 파드가 트래픽 처리
- ▶ **여러 노드에 분산:** pod anti-affinity로 같은 노드 집중 방지
- ▶ **비활성 리더 패턴:** 단일 인스턴스만 동작해야 하는 앱은 leader election 사용
- ▶ **PodDisruptionBudget:** 동시에 죽일 수 있는 파드 수 제한

2) 컨트롤 플레인의 가용성



스토리지

etcd 클러스터링

홀수 개 노드 (3 또는 5), Raft [쿼럼](#) 보장

구성 권장

- 홀수 개 (3 또는 5)
- 3개: 1개 장애 견딤 (소규모 권장)
- 5개: 2개 장애 견딤 (대규모 권장)
- 7개 이상: 합의 비용이 커져 비추천

토폴로지

- **Stacked etcd**: 마스터 노드와 같은 노드에 etcd 실행 (kubeadm 기본). 간단하지만 노드 손실 시 두 컴포넌트 모두 잃음.
- **External etcd**: etcd를 별도 노드 클러스터로. 더 안전하고 확장성 좋음. 운영 비용 증가.

백업

etcd는 **단일 진실의 원천**이므로 정기적인 snapshot 백업이 필수입니다. `etcdctl snapshot save`.

API

API 서버 다중 인스턴스

stateless → 모두 active. LB로 분산

API 서버는 **stateless**합니다. 상태는 etcd에 있으니까요. 그래서 여러 인스턴스를 동시에 띄울 수 있습니다.

구성

- 여러 마스터 노드에 API 서버 인스턴스
- 앞단에 **로드 밸런서** (HAProxy, AWS NLB 등)
- 모든 인스턴스가 동시에 요청 처리

주의

kubeconfig의 server URL은 **LB 주소**를 가리켜야 합니다. 직접 마스터 노드 IP를 가리키면 그 노드가 죽었을 때 끊깁니다.

스케줄러/CM의 Leader Election

동시 동작 시 충돌 → lease로 단일 리더 보장

왜 leader가 필요한가?

스케줄러나 컨트롤러는 **상태를 변경**합니다. 동시에 여러 개가 같은 Pending Pod를 스케줄링하면 충돌이 일어납니다.

구현 방식

K8s 리소스(예전엔 Endpoints/ConfigMap, 지금은 **Lease**)에 "리더는 나"라고 기록합니다. 주기적으로 갱신해서 유지하고, 일정 시간 갱신이 없으면 다른 인스턴스가 차지합니다.

```
# Lease 리소스 예시
kubectl get lease -n kube-system

kube-scheduler          master-1
kube-controller-manager master-2
```

이걸 보면 어느 노드가 현재 리더인지 알 수 있습니다.

Stacked vs External etcd

kubeadm의 두 가지 HA 토폴로지

Stacked etcd 토폴로지 (단순)

```
[Master-1: apiserver + scheduler + ctrl
[Master-2: apiserver + scheduler + ctrl
[Master-3: apiserver + scheduler + ctrl
```

- 장점: 노드 수 적음, 구성 간단
- 단점: 노드 하나 잃으면 etcd와 컨트롤 플레인 둘 다 잃음

External etcd 토폴로지 (강력)

```
[Master-1: apiserver + scheduler + ctrl
[Master-2: apiserver + scheduler + ctrl
[Master-3: apiserver + scheduler + ctrl
↓
[etcd-1] [etcd-2] [etcd-3] (별도 노드)
```

- 장점: 더 강력한 격리, 독립적 확장
- 단점: 노드 수 2배, 운영 복잡

대규모 운영 환경에서는 보통 External을 선택합니다.

Lease API와 Coordinated Leader Election

1판에서는 **Endpoints** 또는 **ConfigMap** 기반 leader election을 설명하지만, 현재는 **coordination.k8s.io/Lease** 리소스가 표준입니다. 더 가볍고 etcd 부하도 적습니다. K8s 1.32+ 에서는 **Coordinated Leader Election**이 alpha로 도입되어 버전 호환성까지 고려한 리더 선출이 가능해집니다.

📖 이 슬라이드의 용어 풀이

스케줄러

"이 파드를 어느 노드에서 실행할까?"만 결정하는 컴포넌트예요. 파드를 직접 만들거나 실행하지는 않아요. 조건에 맞는 노드를 고르는 게 전부예요.

컨트롤 플레인

클러스터의 "두뇌" 역할을 하는 영역이에요. 무엇을 어디서 실행할지 결정하는 관리 컴포넌트(API 서버·etcd·스케줄러·컨트롤러 매니저)가 모여 있어요. 예전엔 "마스터 노드"라고 불렀어요.

쿼럼

"과반수"라는 뜻이에요. 예: 3대 중 2대가 동의해야 결정이 확정돼요. 그래서 etcd는 홀수 개(3,5,7)로 두어 과반을 명확히 만들어요.

API 서버

모든 컴포넌트가 거쳐가는 중앙 창구예요. kubectl·스케줄러·kubelet 등은 서로 직접 대화하지 않고 전부 API 서버를 통해서만 통신해요. (영문: API Server)

ConfigMap

앱 설정값(환경변수, 설정파일 등)을 코드와 분리해 담아두는 리소스예요. 비밀이 아닌 일반 설정에 써요.

Endpoints

Service가 트래픽을 보낼 "실제 파드 IP 목록"이에요. 파드가 늘거나 죽으면 이 목록이 갱신되고, kube-proxy가 그걸 보고 규칙을 다시 써요.

etcd

클러스터의 모든 정보(어떤 파드가 있고 상태가 어떤지 등)를 저장하는 데이터베이스예요. 클러스터의 "공식 장부"라고 보면 돼요. 오직 API 서버만 직접 접근할 수 있어요.

Leader Election

같은 역할의 인스턴스를 여러 개 띄워두되, "지금 일하는 건 한 명"만 정하는 방식이에요. 리더가 죽으면 대기하던 다른 인스턴스가 이어받아요(HA).

Lease

"내가 지금 리더야"를 잠깐 동안 빌려 표시하는 리소스예요. 주기적으로 갱신하고, 끊기면 다른 인스턴스가 가져가요. Leader Election에 써요.

Raft

etcd 같은 분산 시스템에서 여러 서버가 "같은 데이터"에 합의하도록 만드는 알고리즘이에요. 다수결로 리더를 뽑고, 그 리더가 기록을 결정해요.

stateless

자기 안에 상태(데이터)를 저장하지 않는 성질이에요. API 서버는 상태를 전부 etcd에 두기 때문에 여러 개를 동시에 띄워도 괜찮아요.

END 핵심 요약

한 장으로 정리하는 11장의 핵심

🎯 11장을 한 문장으로: 쿠버네티스는 etcd에 적힌 "원하는 상태"를, 여러 컴포넌트가 API 서버를 통해 Watch(알림)하며 "현재 상태"와 끝없이 맞춰가는 분산 시스템이다.

🗣️ 발표 마무리 멘트 예시: "결국 쿠버네티스의 모든 동작은 '원하는 상태와 현재 상태를 비교해서 계속 조정한다'는 한 가지 원리로 설명됩니다. 그래서 컴포넌트 하나가 죽어도 다시 살아나면 알아서 복구되고, 이게 쿠버네티스가 견고한 이유입니다."

🎯 11장 핵심 요약 (발표 마무리용)

- ▶ **쿠버네티스 = 분산 시스템**: 컨트롤 플레인 (etcd/API서버/스케줄러/CM) + 워커 (kubelet/kube-proxy/런타임)
- ▶ **API 서버가 진실의 원천**: 모든 컴포넌트는 API 서버를 통해서만 통신, etcd는 API 서버만 접근
- ▶ **모든 자동화는 Watch + Reconciliation**: 선언된 상태와 현재 상태를 비교해서 끊임없이 조정
- ▶ **파드 = pause 컨테이너 + 앱 컨테이너들**: 네임스페이스 공유의 비밀
- ▶ **K8s 네트워크 모델 = NAT 없는 평평한 네트워크**: CNI 플러그인이 구현
- ▶ **서비스 = iptables(또는 IPVS) 규칙**: ClusterIP는 어디에도 없는 가상 IP
- ▶ **HA = etcd 쿼럼 + 다중 API 서버 + Leader Election**

📖 2판/최신 버전 핵심 차이 요약

중요

Docker → [containerd/CRI-O](#)

K8s 1.24에서 [dockershim](#) 완전 제거

타임라인

- **K8s 1.20 (2020.12)**: dockershim deprecated 공지
- **K8s 1.24 (2022.05)**: dockershim 완전 제거

지금 써야 하는 것

- **containerd**: 가장 널리 쓰임. Docker 내부에서 쓰던 컴포넌트.
- **CRI-O**: K8s 전용으로 설계된 가벼운 런타임. OpenShift 기본.

오해 풀기

"Docker로 빌드한 이미지 못 쓰나요?" → **아니요**. Docker 이미지는 OCI 표준이라 containerd/CRI-O에서 그대로 동작합니다. 단지 노드의 컨테이너 실행 엔진이 바뀐 것일 뿐입니다.

중요

PSP → Pod Security Admission

K8s 1.25에서 PSP 완전 제거

PSP의 문제

- RBAC과 결합이 복잡
- 예측 어려운 정책 머지 동작
- 설정 실수 시 클러스터 전체 마비

Pod Security Admission (PSA)

K8s 1.25+ 부터 내장. 네임스페이스 레이블만으로 정책 적용:

```
apiVersion: v1
kind: Namespace
metadata:
  name: my-app
  labels:
    pod-security.kubernetes.io/enforce:
    pod-security.kubernetes.io/warn: re
    pod-security.kubernetes.io/audit: r
```

3가지 표준

- **privileged**: 무제한 (기본)
- **baseline**: 알려진 권한 상승 방지
- **restricted**: 강력하게 강화 (best practice)

더 정교한 정책이 필요하면 **OPA Gatekeeper**, **Kyverno** 같은 외부 솔루션 사용.

중요

Endpoints → EndpointSlices

대규모 서비스 성능 개선

기존 Endpoints의 문제

서비스 하나당 단일 Endpoints 리소스. 백엔드 파드가 수천 개면 리소스가 거대해지고, 파드 하나 변경할 때마다 **전체**가 다시 전송됩니다.

EndpointSlices

- 서비스 하나당 여러 작은 슬라이스 (기본 100개 씩)
- 변경 시 영향 받는 슬라이스만 갱신
- API 서버, etcd, kube-proxy 모두 부담 감소
- IPv4/IPv6 듀얼 스택 지원 용이

호환성

K8s 1.21+ 부터 기본 활성화. 기존 Endpoints도 호환을 위해 유지되지만, 컨트롤러는 EndpointSlices를 우선 사용합니다.

트렌드

Ingress → Gateway API

L7 라우팅의 차세대 표준

Ingress의 한계

- 표현력 부족 (가중치, 헤더 기반 등 어려움)
- Vendor annotation 난립
- 역할 분리 어려움

Gateway API

K8s 1.29+ 에서 GA. 역할 분리 + 풍부한 라우팅 기능.

```
# 가중치 트래픽 분할 예시 (canary)
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
spec:
  rules:
    - matches:
      - path: { value: /api }
      backendRefs:
        - name: app-v1
          weight: 90
        - name: app-v2
          weight: 10
```

이전 시기

이미 Istio, Contour, Cilium 등 주요 솔루션이 Gateway API를 지원합니다. 신규 프로젝트는 Gateway API로 시작하는 게 좋습니다.

중요

Heapster → [Metrics Server](#)

Heapster는 죽었습니다

Heapster 역사

- 2015: K8s 모니터링 표준으로 출시
- 2018: deprecated 공식 발표
- 이후: GitHub 리포 archived

대체재

- **Metrics Server**: 가벼움. HPA, **kubectl top**용. 영구 저장 X.
- **Prometheus**: 시계열 저장, 알림, 그래프 (Grafana)
- **kube-state-metrics**: 리소스 상태(레플리카 수, 컨테이너 재시작 등)
- **cAdvisor**: 노드/컨테이너 리소스 메트릭 (kubelet에 내장)
- **OpenTelemetry**: 통합 관측성 표준

설치 우선순위

최소: Metrics Server (HPA 위해). 운영: Prometheus + Grafana + Alertmanager 풀스택.

트렌드

iptables → IPVS / [eBPF](#)

대규모 환경에선 IPVS, 차세대는 eBPF(Cilium)

진화 단계

1. **userspace**: 초기, 느림 (deprecated)
2. **iptables**: 1판 기본, O(n) 규칙 검색
3. **IPVS**: 대규모 환경 권장, O(1)
4. **eBPF (Cilium)**: 차세대, kube-proxy 자체 대체 가능
5. **nftables**: K8s 1.29+ beta, iptables 후속

언제 어떤 걸 쓰나?

- ~수백 서비스 → **iptables**로 충분
- 수천 서비스 → **IPVS**
- L7 정책, 옴저버빌리티, 멀티클러스터 → **Cilium(eBPF)**

Cilium은 미래 트렌드입니다. K8s 1.31+ 와 결합해 매우 강력한 네트워킹 + 보안 + 옴저버빌리티 스택을 제공합니다.

🧠 추가로 알아두면 좋은 포인트

- ▶ **etcd가 죽으면?** 새로운 변경은 불가능하지만, 이미 실행 중인 파드는 계속 동작 (kubelet이 캐시된 상태로 운영)
- ▶ **마스터 노드가 모두 죽으면?** 신규 배포/스케줄링은 불가하지만, [워커 노드](#)의 파드는 계속 실행됨
- ▶ **ClusterIP에 ping이 왜 안 되나?** iptables는 TCP/UDP만 [DNAT](#) 함. ICMP는 처리하지 않으므로 ping 실패
- ▶ **Docker가 빠졌는데 기존 이미지는?** OCI 표준이라 그대로 동작. 노드의 런타임만 바뀐 것일 뿐

발표를 마치겠습니다. 들어주셔서 감사합니다 🙏

📖 이 슬라이드의 용어 풀이

네임스페이스

여기서는 리눅스 커널의 격리 기능이에요. 프로세스마다 네트워크·PID 등을 따로 보이게 해서 컨테이너를 서로 격리해요. (K8s의 namespace 리소스와는 다른 개념!)

스케줄러

"이 파드를 어느 노드에서 실행할까?"만 결정하는 컴포넌트예요. 파드를 직접 만들거나 실행하지는 않아요. 조건에 맞는 노드를 고르는 게 전부예요.

워커 노드

실제로 앱 컨테이너가 돌아가는 일꾼 서버예요. 컨트롤 플레인인 "이 파드를 띄워"라고 시키면 여기서 실행돼요.

컨트롤 플레인

클러스터의 "두뇌" 역할을 하는 영역이에요. 무엇을 어디서 실행할지 결정하는 관리 컴포넌트(API 서버·etcd·스케줄러·컨트롤러 매니저)가 모여 있어요. 예전엔 "마스터 노드"라고 불렀어요.

쿼럼

"과반수"라는 뜻이에요. 예: 3대 중 2대가 동의해야 결정이 확정돼요. 그래서 etcd는 홀수 개(3,5,7)로 두어 과반을 명확히 만 들어요.

API 서버

모든 컴포넌트가 거쳐가는 중앙 창구예요. kubectl·스케줄러·kubelet 등은 서로 직접 대화하지 않고 전부 API 서버를 통해서만 통신해요. (영문: API Server)

cAdvisor

kubelet에 내장된, 노드와 컨테이너의 리소스 사용량을 수집하는 도구예요.

ClusterIP

Service의 기본 유형. 클러스터 "내부에서만" 접근 가능한 가상 IP를 줘요. 실제 서버가 아니라 iptables 규칙으로 만들어진 가짜 IP예요.

CNI

Container Network Interface. kubelet이 파드를 만들 때 "IP 할당, 네트워크 연결"을 위임하는 플러그인 표준이에요. Calico·Cilium·Flannel 등이 구현체예요.

containerd

가장 널리 쓰이는 컨테이너 런타임이에요. 원래 Docker 내부에서 쓰던 엔진이 독립한 것. Docker로 만든 이미지도 그대로 실행돼요.

CRI

Container Runtime Interface. kubelet과 컨테이너 런타임 사이의 "표준 약속"이에요. 이 규격만 맞추면 어떤 런타임이든 K8s에 끼울 수 있어요.

CRI-O

쿠버네티스 전용으로 가볍게 설계된 컨테이너 런타임이에요. OpenShift의 기본 런타임이에요.

DNAT

Destination NAT. 패킷의 "목적지 IP"를 바꿔치기하는 거예요. Service의 가상 IP로 온 패킷의 목적지를 실제 파드 IP로 바꿔 전달할 때 써요.

dockershim

예전에 kubelet이 Docker를 호출하려고 두었던 중간 어댑터예요. K8s 1.24에서 제거됐고, 지금은 containerd/CRI-O를 직접 써요.

eBPF

리눅스 커널 안에서 안전하게 작은 프로그램을 실행하는 기술이에요. Cilium이 이걸로 kube-proxy를 대체하고 빠른 네트워킹/보안을 제공해요.

Endpoints

Service가 트래픽을 보낼 "실제 파드 IP 목록"이에요. 파드가 늘거나 죽으면 이 목록이 갱신되고, kube-proxy가 그걸 보고 규칙을 다시 써요.

EndpointSlices

Endpoints의 개선판. 파드가 수천 개여도 작은 조각(슬라이스)으로 쪼개 관리해서, 하나 바뀔 때 전체가 아니라 해당 조각만 갱신해요.

etcd

클러스터의 모든 정보(어떤 파드가 있고 상태가 어떤지 등)를 저장하는 데이터베이스예요. 클러스터의 "공식 장부"라고 보면 돼요. 오직 API 서버만 직접 접근할 수 있어요.

Gateway API

Ingress의 후속 표준이에요(K8s 1.29+ GA). 가중치 분할·헤더 라우팅 등 표현력이 풍부하고, 인프라/네트워크/앱 팀의 역할을 나눠 관리해요.

HPA

Horizontal Pod Autoscaler. CPU/메모리 사용량 등을 보고 파드 개수를 자동으로 늘리거나 줄여주는 기능이에요.

Ingress

클러스터 밖에서 들어오는 HTTP(L7) 트래픽을 URL 경로·호스트명 기준으로 여러 Service에 나눠 보내는 라우팅 규칙이에요.

iptables

리눅스 커널의 패킷 필터링/변환 규칙 시스템이에요. kube-proxy가 여기에 규칙을 넣어 Service IP로 온 트래픽을 실제 파드로 보내요.

IPVS

리눅스 커널의 로드밸런싱 기능이에요. iptables보다 규칙이 많아도 빨라서(O(1)) 서비스가 수천 개인 대규모 환경에 권장돼요.

kube-proxy

각 노드에서 Service(가상 IP)로 온 트래픽을 실제 파드로 전달되도록 네트워크 규칙(iptables 등)을 관리하는 컴포넌트예요.

Leader Election

같은 역할의 인스턴스를 여러 개 띄워두되, "지금 일하는 건 한 명"만 정하는 방식이에요. 리더가 죽으면 대기하던 다른 인스턴스가 이어받아요(HA).

Metrics Server

CPU/메모리 사용량을 가볍게 모아주는 컴포넌트예요. kubectl top과 HPA가 이걸 써요. 장기 저장은 안 해요.

pause 컨테이너

파드마다 몰래 하나씩 뜨는 거의 아무것도 안 하는 컨테이너예요. 파드의 네트워크/IPC 공간을 처음 만들어 "붙잡고" 있어서, 앱 컨테이너가 재시작돼도 파드 IP가 유지돼요.

Prometheus

시계열(시간별) 메트릭을 저장하고 알림·그래프를 제공하는 모니터링 도구예요. Grafana와 함께 많이 써요.

RBAC

Role-Based Access Control. "역할(Role)"에 권한을 묶고, 사용자/서비스에 역할을 부여해 권한을 관리하는 방식이에요. K8s의 표준 인가 방법이에요.

Watch

API 서버가 "리소스가 바뀌면 알려줄게"라고 변경 사항을 실시간으로 밀어주는(push) 기능이에요. 계속 물어보는 폴링보다 효율적이에요.